LFS256: DevOps and Workflow Management with Argo

v011.07.2025

Lab 3.1. Installing Argo CD

In Chapter 3, we reviewed the fundamentals of Argo, exploring its capabilities and how it revolutionizes Kubernetes cluster management through GitOps. Building on that foundation, this lab marks the beginning of your hands-on journey with Argo CD, a crucial component of the Argo ecosystem.

In this lab, we'll be transitioning from theory to action. Here, you will deploy Argo CD directly into a Kubernetes cluster. This exercise is designed to provide you with practical experience and a deeper understanding of Argo CD's operational mechanics. By the end of this lab, you will have a fully functional Argo CD instance running in your cluster, serving as a cornerstone for the advanced scenarios we'll tackle in subsequent labs.

Objective

Install Argo CD, and gain access to both the Argo CD web UI and CLI.

Prerequisites

- Basic understanding of Docker, Kubernetes, and command-line interface operations.
- Access to a computer with an internet connection.
- A running Kubernetes cluster.
    - See Chapter 2's "Deploying Kubernetes for Argo" section for details on how to set up a cluster.

Deploy Argo CD to Kubernetes

Create a namespace for Argo with the following command:

```
$ kubectl create namespace argocd
```

Deploy ArgoCD using the quick start manifest with the following command:

```
$ kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/in
stall.yaml
```

Verify that Argo CD is installed by running the following command:

```
$ kubectl get pods -n argocd
```

Accessing Argo CD

ArgoCD offers an intuitive and user-friendly web-based interface that simplifies the management of applications deployed on Kubernetes clusters. Below, we will explain access via the UI. Access via the CLI is also possible but explained in detail in Chapter 6.

Login via UI

In order to access Argo CD's API, you need to port forward the service. This should be done in a separate terminal window so you can run commands from your current terminal window.

Command:

```
$ kubectl port-forward svc/argocd-server -n argocd 8080:443 --address
0.0.0.0
```

Output:

```
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

Argo CD comes with a built-in admin user and auto-generated password. The password is stored as a Kubernetes secret which can be retrieved with the following command in a new terminal (as the port-forward may have taken control of your previous session):

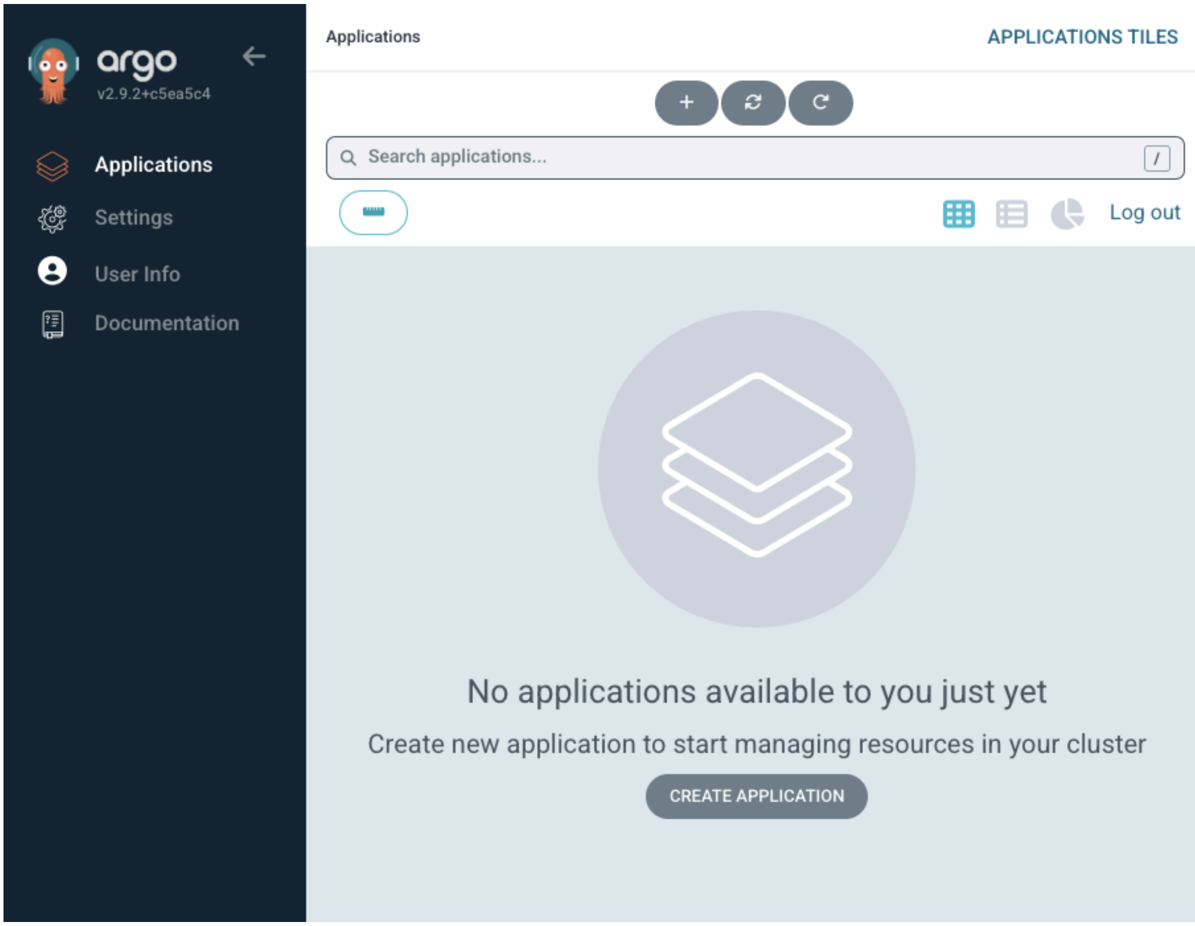
```
$ kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d; echo
```

Access the UI on https://localhost:8080 (or your VM's public IP on port 8080). Due to the self-signed certificate, you will receive a TLS error which you will need to manually approve.

Pay close attention to the URI. It uses HTTPS and not HTTP. Navigating to http://localhost:8080 will result in a server-side error that breaks the port forwarding.

On the UI, you can login with the user **`admin`** and the auto-generated password.

Once logged in, you should see the following screen:



Screenshot of Argo CD UI after logging in (your version may differ)

Login via CLI

Download and install the [ArgoCD CLI]() tool suitable for your operating system and corresponding to your specific ArgoCD release version. More detailed installation instructions can be found in the [CLI installation documentation]().

For Ubuntu 24.04, you can install the latest (as of this publication) ArgoCD CLI using the following commands:

```
$ wget
https://github.com/argoproj/argo-cd/releases/download/v3.1.8/argocd-li
nux-amd64 -O argocd
```

```
$ chmod +x argocd
```

```
$ sudo mv argocd /usr/local/bin/
```

For users on Mac, Linux, and WSL who prefer using Homebrew, you can install the tool by running the following command:

```
$ brew install argocd
```

Using the username admin, the auto-generated password from above, and the port-forwarded service, log in to Argo CD accessible at https://localhost:8080.

Command:

```
$ argocd login localhost:8080 --username admin --password <The Admin
Password from before>
```

Output:

```
WARNING: server certificate had error: tls: failed to verify
certificate: x509: certificate signed by unknown authority.
Proceed insecurely (y/n)? y
Username: admin
Password:
'admin:login' logged in successfully
Context 'localhost:8080' updated
```

Upon successful login, you should see a message indicating that you're logged in to the specified ArgoCD server URL.

Lab 3.2. Managing Applications with Argo CD

In this lab, we're moving beyond the basics and getting into the practical implementation of Argo CD, showcasing its robust capabilities in deploying applications into a Kubernetes cluster. This session is tailored to reinforce your understanding of Argo CD by offering a hands-on experience in a real-world setting.

You'll not only deploy an application using Argo CD and GitOps principles but also learn how to modify the deployed application. Witness firsthand how Argo CD swiftly updates pods in response to changes, and understand the process of rolling back to a previously deployed version, ensuring a smooth and reliable deployment experience.

Objective

Learn to deploy an application to Kubernetes using Argo CD and GitOps principles. In this lab, you will deploy an example application.

Prerequisites

- Argo CD already installed and running in your Kubernetes cluster.
    - See Chapter 2's "Deploying Kubernetes for Argo" section for details on how to set up a cluster
- Basic understanding of Kubernetes and Argo CD.
- Git hosting service (GitHub recommended).

Deploy the Application

1. Fork the GitHub repository

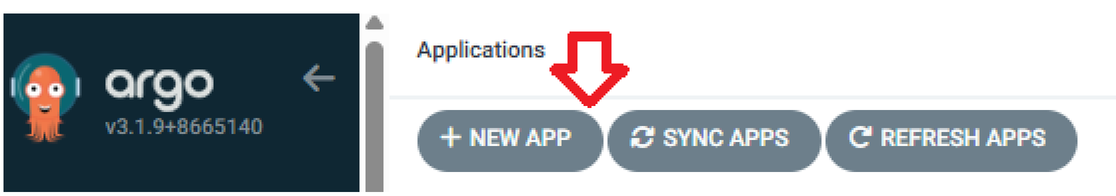
To begin, fork the following GitHub repository: https://github.com/lftraining/LFS256-code. This step is crucial as it allows you to have your own copy of the application, enabling you to make and track changes independently during the lab.

Before you proceed in your fork, change the **`targetPort`** value in the **`argocd/example-app/service.yaml`** file to 80:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: argocd-example-app-service
spec:
  type: NodePort
  selector:
    app: argocd-example-app
  ports:
    - port: 3000
      targetPort: 80
      nodePort: 31000 # Optional: Kubernetes will choose a port if this is omitted
```

2. Create the application in Argo CD

To create an Application log into the UI again by accessing https://localhost:8080 and select "+ New App".

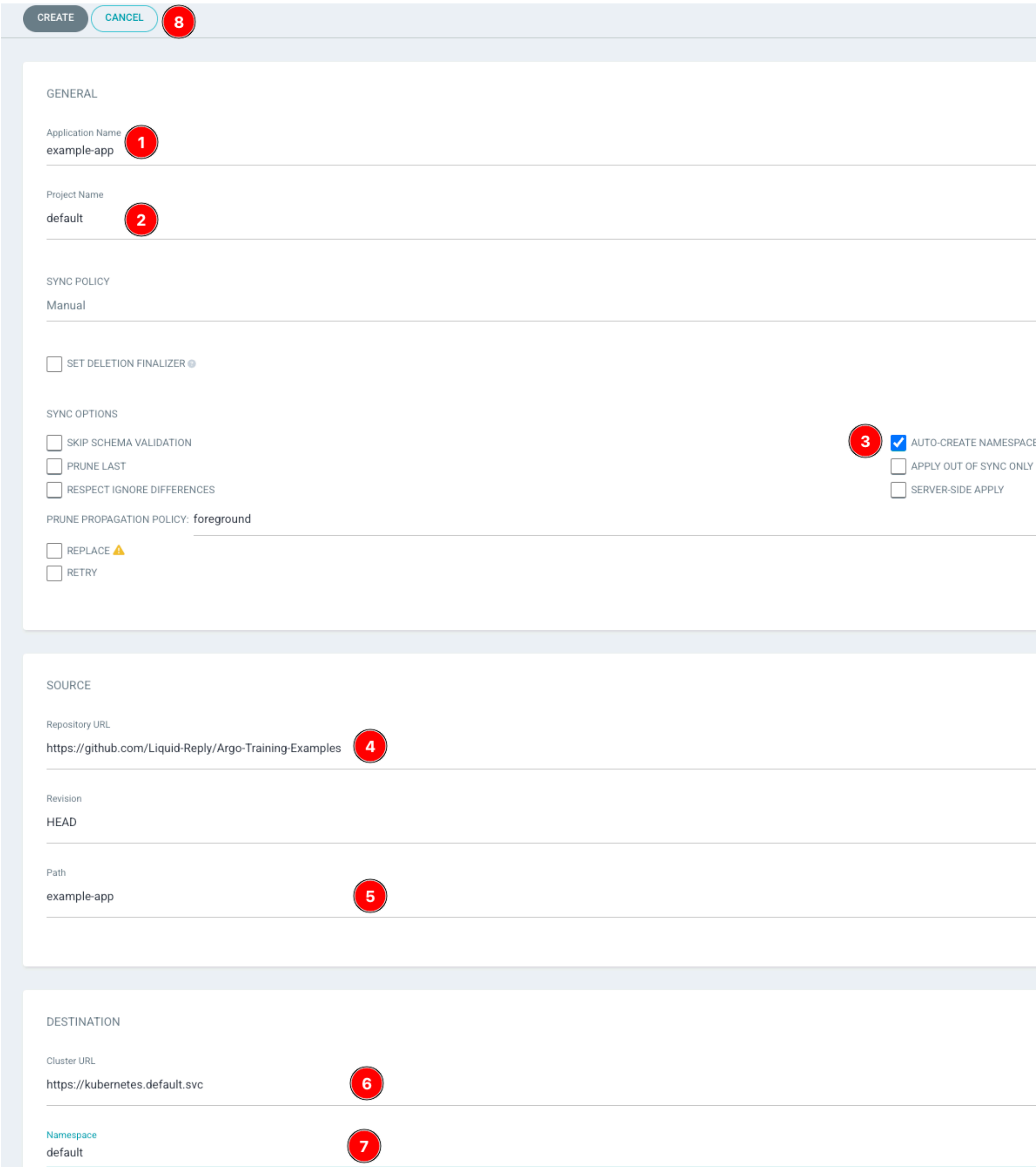


Start the process of creating a new application

Next,

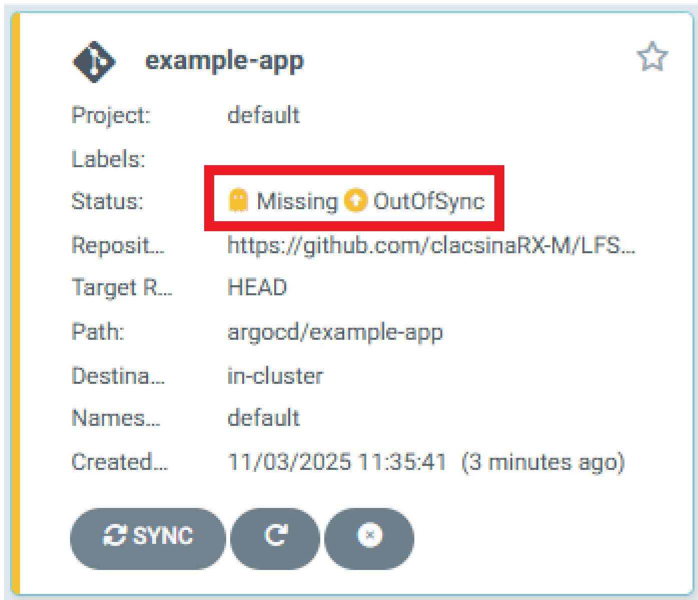
1. Write a name of your choice in "Application Name" (e.g., "example-app").
2. As Project Name choose "default".
3. Under Sync Options tick "Auto-Create Namespace".

4. In the “Source” section paste in “Repository URL” (the URL of the repository you forked in the step before).
5. In “Path” name the directory of the repository in which the YAML files are located. In this example, it will be “argocd/example-app”.
6. Under the “Destination” section define the “Cluster-URL”. In this example we use “https://kubernetes.default.svc”.
7. Define a namespace of your choice (e.g., “default”).
8. Select the “Create” button at the top.



Process of creating a new application

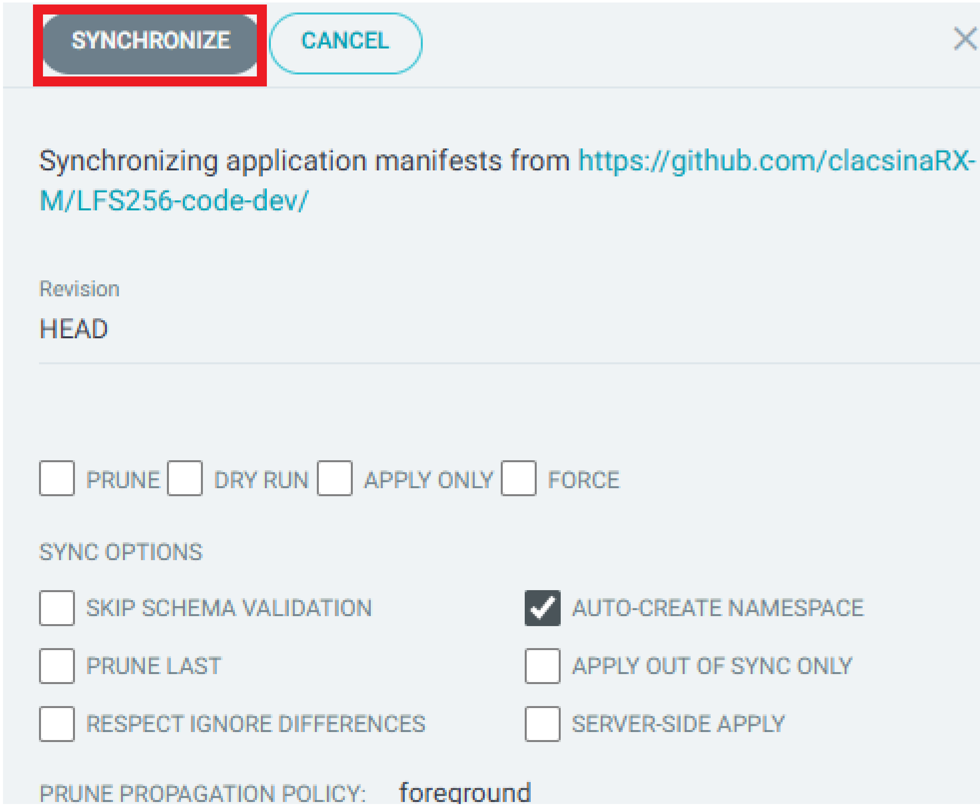
After creating the application in Argo CD, check its status using the UI. Note that the application status will be `OutOfSync` as it's yet to be deployed.



Undeployed application of Argo CD

3. Deploy the application

Select the “sync” button. This will open a new window. In that window you have several options to synchronize the application. For now we only have to choose “Synchronize”. The status should change to Synced and Progressing or Healthy.



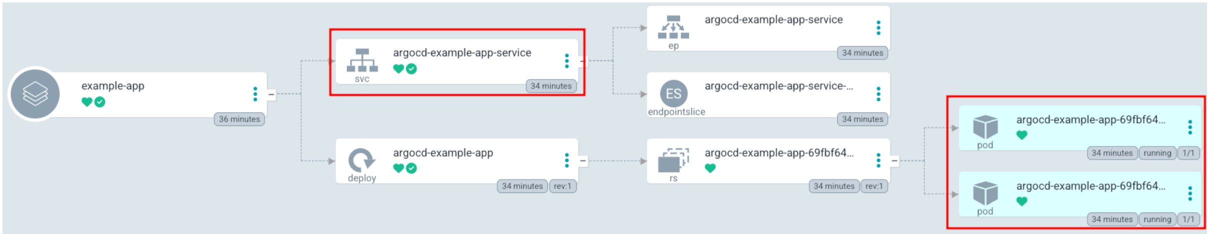
Synchronizing an application

4. Access the deployed application

Select the newly created application. In the ensuing window, you'll gain insights into its resource components, such as the two pods displayed on the right. The deployment configuration, as defined in the provided `deployment.yaml` file, specifies `replicas: 2`, indicating a desired state of two identical pods running concurrently.

Additionally, you can access logs, review events, and obtain a more detailed view of the current health status.

Make sure to take note of the associated service named `argocd-example-app-service`. This service is crucial for accessing the application through your browser. Activate port forwarding for this service to seamlessly interact with the application in your local browser environment.



Created resources for the new application

You can forward the port of the service by running the following command:

```
$ kubectl port-forward svc/argocd-example-app-service 9090:3000
--address 0.0.0.0
```

Output:

```
Forwarding from 127.0.0.1:9090 -> 3000
Forwarding from [::1]:9090 -> 3000
```

Open http://localhost:9090 in your browser to view the application. You should see on your screen “Application Version: 1.0.0”.

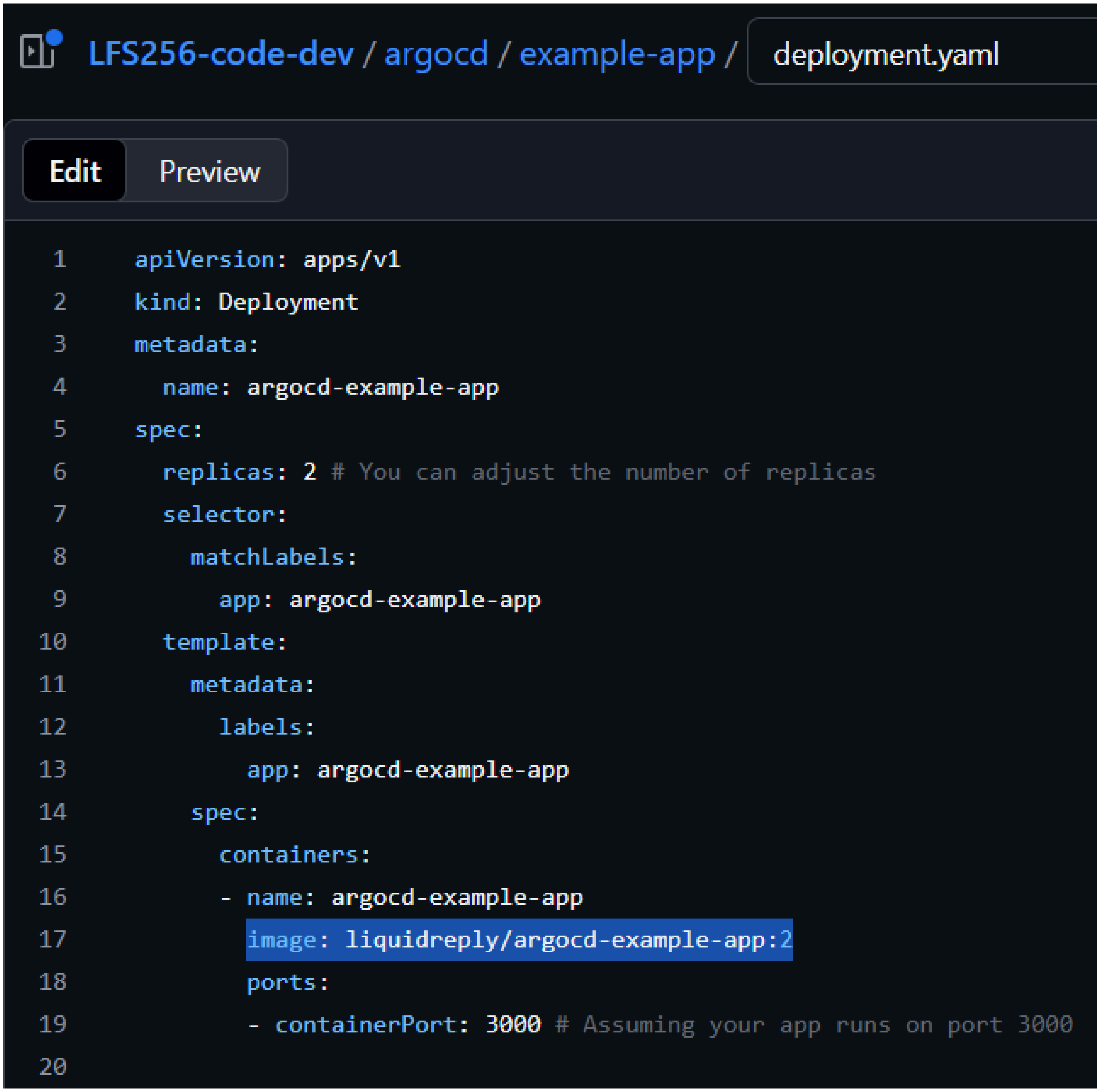
Created application in the browser

From another terminal, you can also access the application by using curl with this command:

```
$ curl localhost:9090

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>ArgoCD is great!</title>
</head>
<body>
    <p>Application Version: 1.0.0</p>
</body>
</html>
```

5. Update the Image

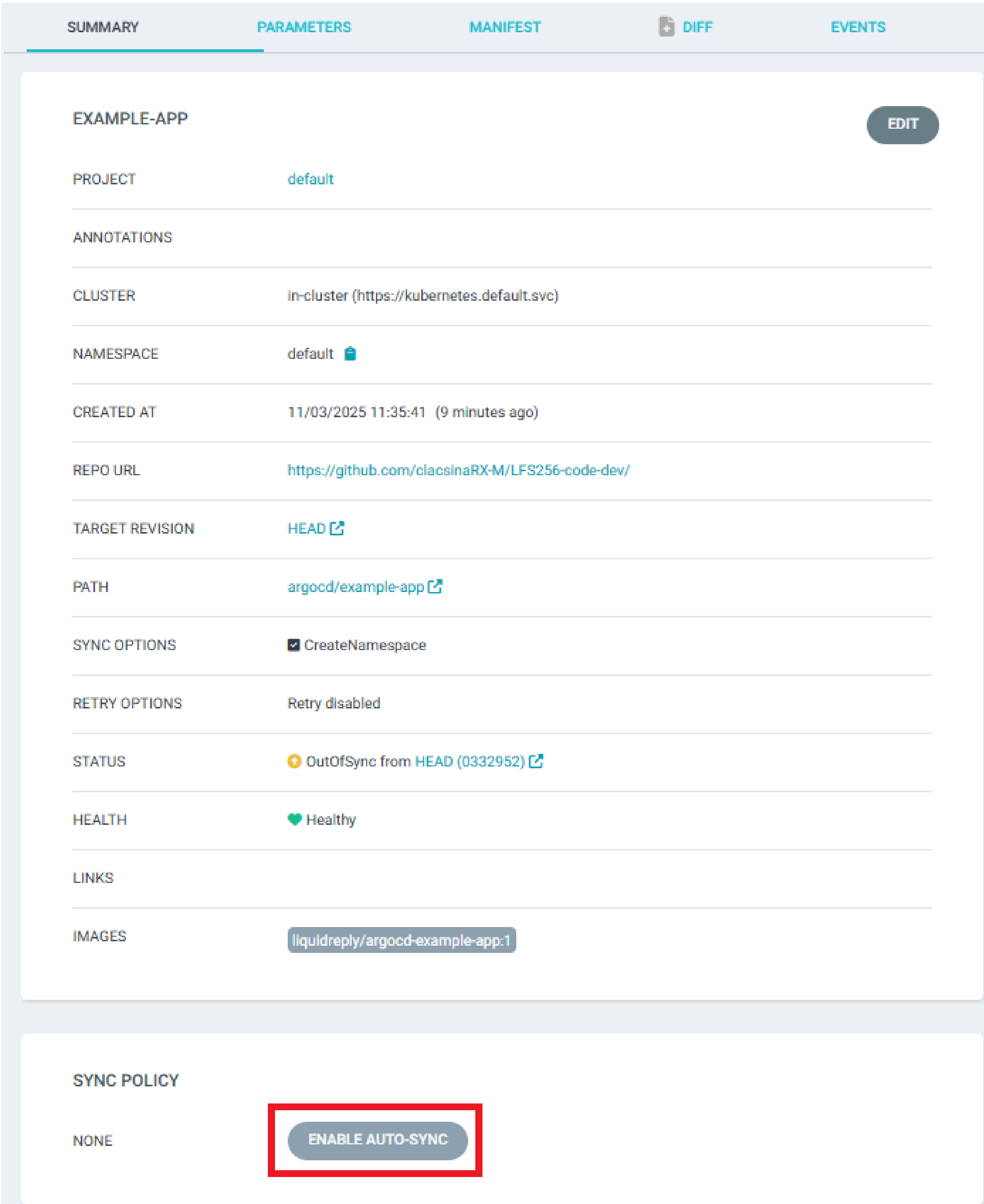
In your forked repository, navigate to the deployment YAML file (`deployment.yaml`). Locate the section where the image is specified, and change the tag to a new version. If the current image tag is **`liquidreply/argocd-example-app:1`**, update it to **`liquidreply/argocd-example-app:2`**.



Commit and push the changes.

In your UI, press “Refresh”. You will see the status is now **`OutOfSync`**. To sync it, repeat step 4. It's also possible to activate AutoSync.

Choose the application. At the top left, select “App Details”. In the new window, you will find a button “Enable Auto-Sync”. Choose it and confirm it.



If you now change the image again or anything else in the yaml file it will automatically sync the application.

After synchronization you have to restart the port forwarding. Open http://localhost:9090 and you should be able to see “Application Version: 2.0.0”.

Updated application in the browser

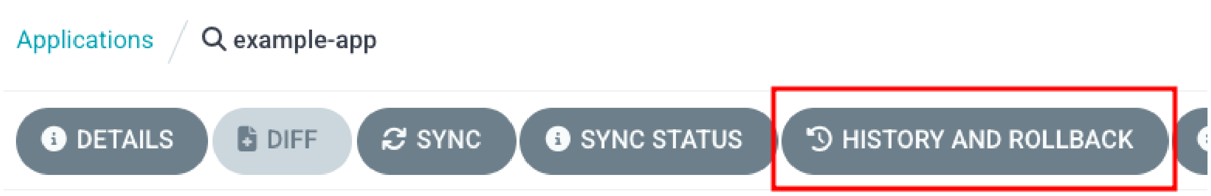
From another terminal, you can also access the application by using curl:

```
$ curl localhost:9090

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>ArgoCD is great!</title>
</head>
<body>
    <p>Application Version: 2.0.0</p>
</body>
</html>
```

6. Rollback

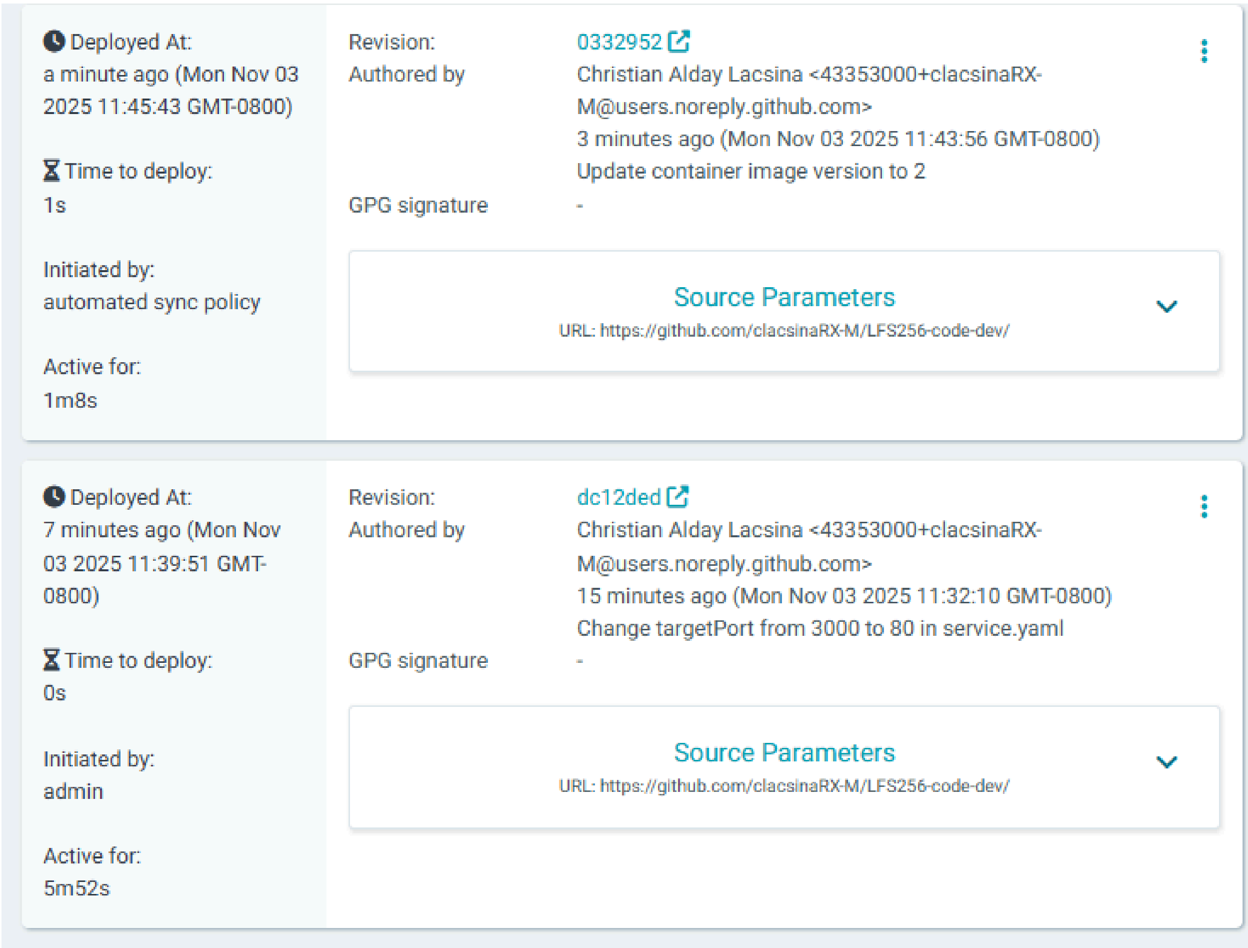
Now, let's delve into Argo CD's rollback capability. After updating your application, we'll guide you through effortlessly reverting to the previous version.

In the UI, you will find inside the application a button labeled “History and Rollback”.

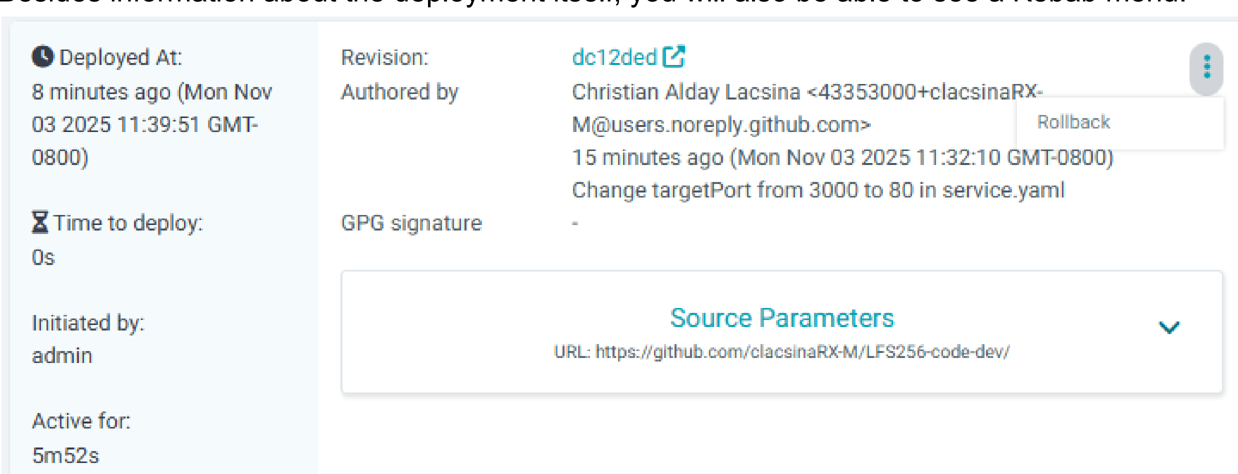


Menu of an application

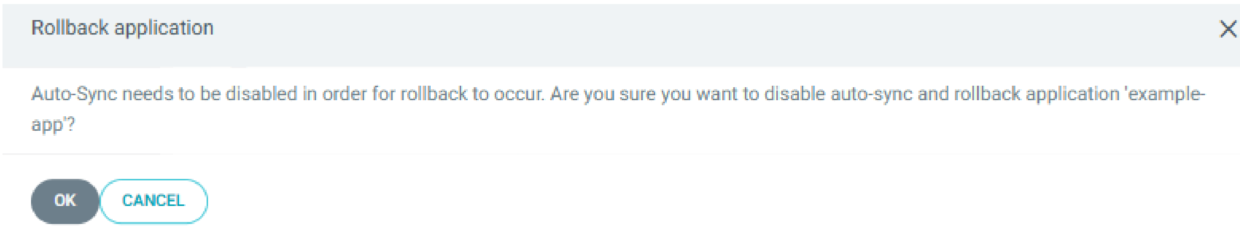
If you select it, a new window will open. In this window, you can see the history of deployment for your application.



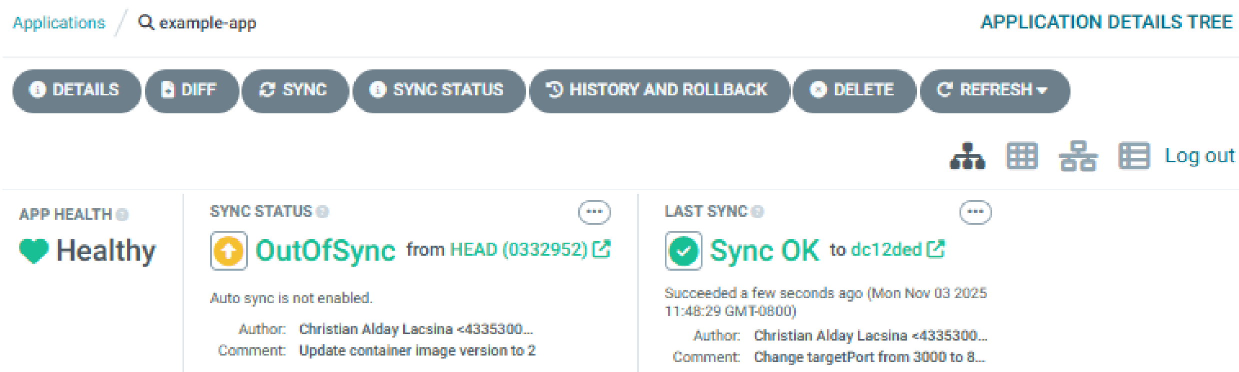
Besides information about the deployment itself, you will also be able to see a Kebab menu.



Select it and choose to rollback to that deployment. If you have Autosync on, you will be prompted to turn it off for the rollback to occur:



After a few moments, the app will report that it is out of sync:



Redo the kubectl port-forward and try accessing your app again:

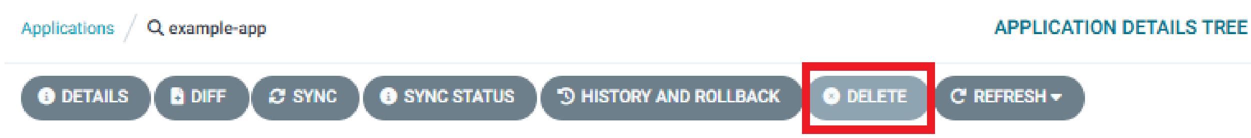
```
$ kubectl port-forward svc/argocd-example-app-service 9090:3000
--address 0.0.0.0

$ # in another terminal

$  curl localhost:9090

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <title>ArgoCD is great!</title>
```

```
</head>

<body>

    <p>Application Version: 1.0.0</p>

</body>

</html>
```

7. Deleting An Application

At some point, you will have to retire an active application. Argo CD makes this easy by tying all of the assets of an app in Kubernetes (e.g. the deployments, services, etc.) to the existence of its associated Application custom resource.

In your UI, click “Delete” in the application’s toolbar:

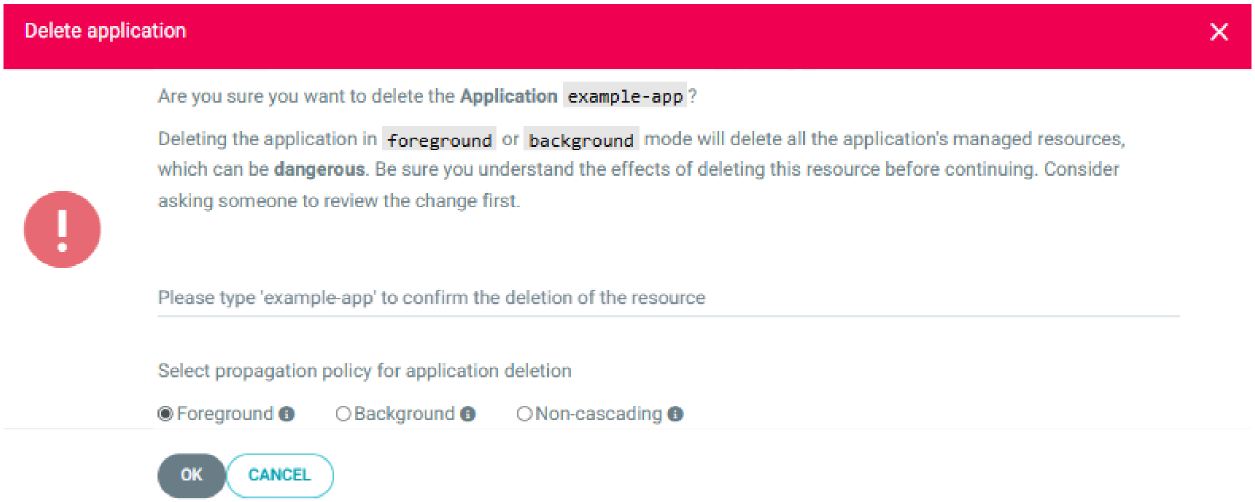


You will be prompted to confirm the deletion.

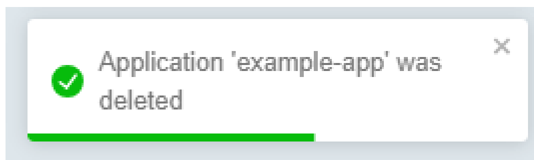
It describes some of the things to consider, such as the deletion policy.

There are 3 policies available:

- Foreground propagation - which has Argo CD terminate all child resources before the parent.
- Background propagation - which has ArgoCD delete the parent first and lets Kubernetes remove the child objects asynchronously as kubectl normally would.
- Do not touch the resources at all - which will keep the Kubernetes resources but remove it from Argo CD’s management.



Type your app's name and click OK with the foreground policy enabled.



Lab 3.3. Argo CD Security and RBAC

Security is paramount, and in this lab, we explore how to configure authentication and authorization in Argo CD. Implement Role-Based Access Control (RBAC) to finely manage user permissions.

Objective

Learn how to finely manage user permissions and ensure secure access to Argo CD resources.

Prerequisites

- Argo CD already installed and running in your Kubernetes cluster.
- Basic understanding of Kubernetes and Argo CD.

Configure Authentication and Authorization

1. Create a new user

Open the ConfigMap for editing:

```
kubectl edit cm argocd-cm -n argocd
```

Inside the ConfigMap, find or create the **data** section, then add a new user:

```
data:
  accounts.developer: login
```

The user "Developer" has now been created. To log in, it needs a password. You can configure it by using the Argo CD CLI.

You can install it on Ubuntu 24.04 by running the following commands:

```
wget
https://github.com/argoproj/argo-cd/releases/download/v3.1.8/argocd-linux-amd64 -O argocd
```

```
chmod +x argocd
```

```
sudo mv argocd /usr/local/bin/
```

Or you can install it with homebrew by running this command:

```
brew install argocd
```

Make sure to log in prior to continuing. First, retrieve your password with the following command:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d; echo
```

Then use the password to log in. The **`localhost:8080`** address assumes you have an active **`kubectl port-forward`** for the Argo CD on your cluster. Run the following command:

```
argocd login localhost:8080
```

Then set the password with this command (you will be prompted to enter the password for the admin user you just logged in with):

```
argocd account update-password --account developer --new-password
Developer123
```

2. Define the RBAC rules

Open the ConfigMap for editing RBAC rules with this command:

```
kubectl edit cm argocd-rbac-cm -n argocd
```

In here we want to create a new policy:

```
p, role:synconly, applications, sync, */*, allow
g, developer, role:synconly
policy.default: role:readonly
```

This policy defines the following:

- **`p, role:synconly, applications, sync, */*, allow`**: This line specifies a permission rule that allows only the **`sync`** action on applications. The role **`synconly`** is created for this purpose.

- **`g, developer, role:synconly`**: Associates the **`developer`** group with the role **`synconly`**, granting them permission only to sync applications.
- **`policy.default: role:readonly`**: Specifies the default policy for users who don't fall into any explicitly defined category. In this case, they have read-only access.

In Argo CD, the RBAC rules can be intricately defined, allowing for granular control over access permissions. For a deeper understanding and detailed information on RBAC configurations, refer to the official documentation.

In the configmap, app the following data section:

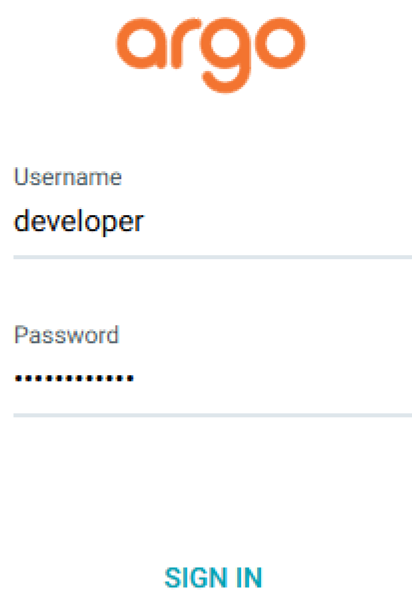
```
data: # Add this data section.
  policy.csv: |
    p, role:synconly, applications, sync, */*, allow
    g, developer, role:synconly
  policy.default: role:readonly
```

3. Test the new rules

If you deleted the **`example-app`** in lab 3.2, recreate as your admin user with the following details:

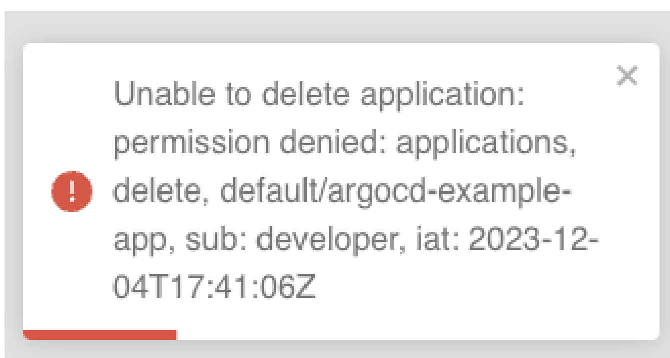
- A name of your choice in “Application Name” (e.g., “example-app”).
- As Project Name choose “default”.
- Under Sync Options tick “Auto-Create Namespace”.
- In the “Source” section paste in “Repository URL” (the URL of the repository you forked in the step before).
- In “Path” name the directory of the repository in which the YAML files are located. In this example it will be “argocd/example-app”.
- Under the “Destination” section define the “Cluster-URL”. In this example we use “https://kubernetes.default.svc”.
- Define a namespace of your choice (e.g., “default”).
- Select the “Create” button at the top.

Now attempt to log in as the 'developer' user and evaluate the access permissions.



Login screen

Verify if you can successfully synchronize applications. This operation should complete successfully. Additionally, explore whether actions such as deletion or other operations are restricted according to the RBAC rules assigned to the 'developer' subject.



Error after denied permission

Lab 4.1. Installing Argo Workflows

In the previous chapter, we discussed the fundamentals of Argo Workflow, exploring its capabilities. A workflow engine designed for Kubernetes, enabling the orchestration of complex tasks and processes within containerized environments. It provides a powerful and flexible framework for defining, running, and managing workflows as a series of interconnected steps. In this chapter, we will focus on installation, accessing the graphical user interface, installing CLI, and finally exploring a few of Argo Workflow's native Kubernetes resources.

Objective

Install Argo Workflows and understand how to access Argo Workflows resources and Argo UI.

Prerequisites

- Basic understanding of Docker, Kubernetes, and command-line interface operations.
- Access to a computer with an internet connection.
- An installation of Kubernetes that you have full control over
    - See Chapter 2's "Deploying Kubernetes for Argo" section for details on how to set one up for yourself

Install Cluster and Argo Workflows

1. Deploy Argo Workflows

Argo Workflows is a separate project under the Argo family of tools. In previous labs, you may have installed Argo CD in its own namespace. While you can install all of the Argo tools under a common Kubernetes namespace, it is best to keep each tool like Workflows, Rollouts, and ArgoCD in their own namespaces. They work very well together, but can work separately.

Create a namespace for Argo Workflows with the following command:

```
kubectl create namespace argo
```

Deploy Argo Workflows using the quick start manifest. Run the following command:

```
kubectl apply -n argo -f
https://github.com/argoproj/argo-workflows/releases/download/v3.7.3/in
stall.yaml
```

Verify that Argo Workflows is installed by running the following command:

```
$ kubectl -n argo get pods
```

Output:

```
NAME                                 READY   STATUS    RESTARTS
AGE
argo-server-5fdfcf6fd6-tfx6m         1/1     Running   0
2m
workflow-controller-5df59665c9-zz92b   1/1     Running   0
2m
```

2. Accessing UI Dashboard

Patch argo-server authentication

The Argo-server (and thus the UI) defaults to client authentication, which requires clients to provide their Kubernetes bearer token to authenticate. For more information, refer to the Argo Server Auth Mode documentation. We will switch the authentication mode to the server so that we can bypass the UI login for now. Run the following command:

```
kubectl patch deployment \
  argo-server \
  -n argo \
  --type='json' \
  -p='[{"op": "replace", "path":
"/spec/template/spec/containers/0/args", "value": [
  "server",
  "--auth-mode=server"
]}]'
```

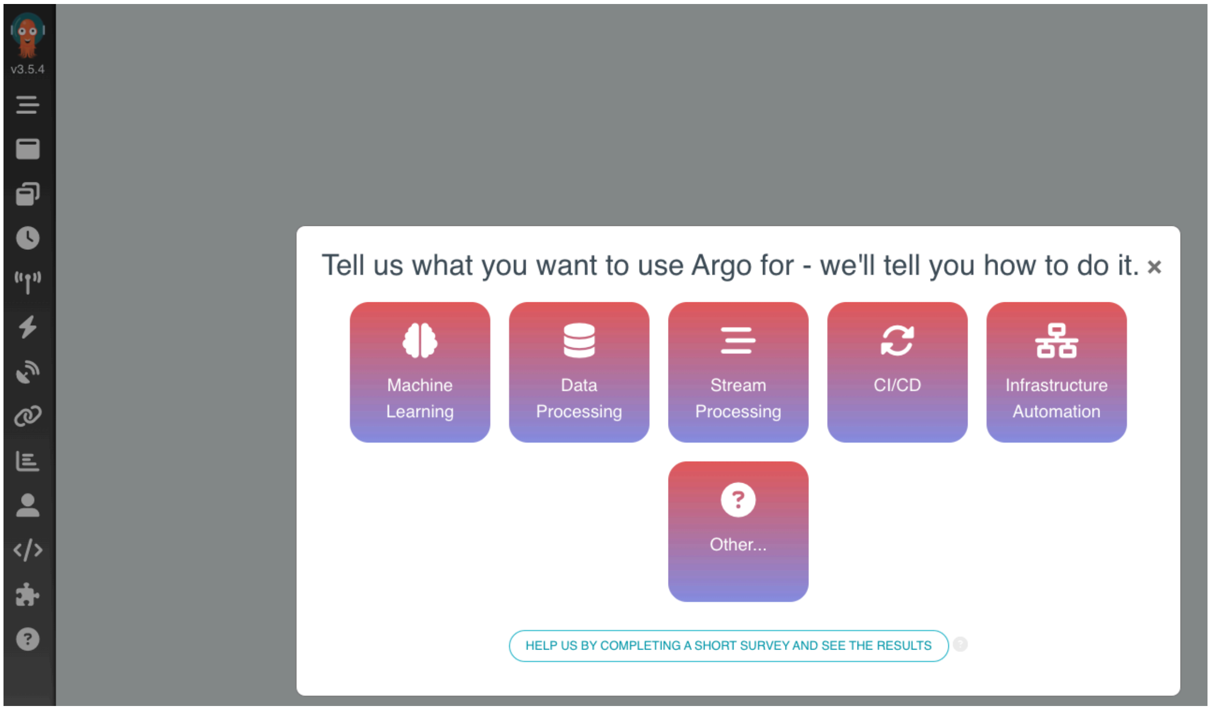
Port-forward the UI

Now you can use port-forward, so you can access the UI. Run the following command:

```
kubectl -n argo port-forward deployment/argo-server 2746:2746
--address 0.0.0.0
```

The Argo Workflows UI will be accessible at https://localhost:2746. Since we are using a self-signed certificate, you need to bypass a TLS error.

Please use **HTTPS** instead of **HTTP** protocol. while navigating to Argo Workflow UI, otherwise it will cause a server-side error that will break the port forwarding.



Start Screen of Argo Workflows

3. Install the Argo Workflows CLI

Download the latest Argo Workflows version from releases. More detailed installation instructions can be found via the CLI installation documentation.

On Ubuntu 24.04, you can install the CLI with the following commands:

```
$ wget
https://github.com/argoproj/argo-workflows/releases/download/v3.7.3/ar
go-linux-amd64.gz -O argo.gz
```

```
$ gunzip "argo.gz"
```

```
$ chmod +x argo
```

```
$ sudo mv argo /usr/local/bin/
```

After doing so, verify that the **argo** CLI is installed correctly by running the following command:

```
$ argo version
```

Output:

```
argo: v3.7.3
  BuildDate: 2025-10-14T11:39:00Z
  GitCommit: bded09fe4abd37cb98d7fc81b4c14a6f5034e9ab
  GitTreeState: clean
  GitTag: v3.7.3
  GoVersion: go1.24.7
  Compiler: gc
  Platform: linux/amd64
```

4. Optional: Enable Shell auto-completion

To get easy access to Argo Rollout Resources, the CLI can add shell completion code for several shells. For bash, you can use the following command:

```
source <(argo completion bash)
```

Other shells are also supported. Please refer to the [completion command documentation] for more details.

5. Using Argo Workflows

The **argo** CLI tool is primarily for the Argo Workflows management via the CLI. It works similarly to the Argo CD CLI in that it can either use the Kubernetes API or work directly with an Argo Server.

This lab will primarily use the web GUI.

More details are listed in the **argo help** output:

```
$ argo help
```

Output:

```
You can use the CLI in the following modes:

#### Kubernetes API Mode (default)

Requests are sent directly to the Kubernetes API. No Argo Server is
needed. Large workflows and the workflow archive are not supported.

Use when you have direct access to the Kubernetes API, and don't need
large workflow or workflow archive support.
```

Here is a list of the most common commands to operate with Argo Workflows:

```
argo list # list workflows
argo get <workflow-name> # display details about a workflow
argo submit <workflow-file.yaml> # submit a workflow
argo watch <workflow-name> # watch workflow progress
argo template list # list workflow templates
argo logs <workflow-name> # display logs for workflow pod(s)
```

Lab 4.2. A Simple DAG Workflow

In this chapter, we begin our hands-on journey with Argo Workflow, creating simple workflows. This lab focuses on practical implementation, allowing users to familiarize themselves with Argo Workflow by deploying a workflow. This step-by-step exploration will create a minimal workflow demonstrating a DAG **(Directed-Acyclic Graph)** Workflow. DAGs are particularly useful in scenarios where tasks or steps have interdependencies, and their execution order needs to be carefully managed. DAGs can be highly beneficial in data processing pipelines, scientific workflows, machine learning, and multi-step deployment workflows. This example will help you understand how to define and run a simple DAG Workflow.

Objective

To see Argo Workflows in action:

- We will create a simple WorkflowTemplate that represents a task in our DAG Workflow, and
- A Workflow that uses the template and defines the relationship between the tasks.

Prerequisites

- Kubernetes cluster with argo-workflow controller.
- Argo Workflows CLI (optional).

Deploy a DAG Workflow

1. Install Resources

Here is an Argo WorkflowTemplate that runs a container. Run the following command to create a manifest called `dag-workflow-template.yaml`:

```
cat <<EOF > dag-workflow-template.yaml
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: echo-template
  namespace: argo
spec:
  serviceAccountName: argo
  templates:
  - name: echo
    inputs:
      parameters:
      - name: message
    container:
      image: alpine:3.7
      command: [echo, "{{inputs.parameters.message}}"]
EOF
```

Create a DAG workflow that uses the template and defines the relationships between tasks. Save this workflow in a file called, `dag-workflow.yaml` as shown below:

```
cat <<EOF > dag-workflow.yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: dag-diamond-
  namespace: argo
spec:
  entrypoint: diamond
  templates:
  - name: diamond
    dag:
      tasks:
      - name: A
        arguments:
          parameters: [{name: message, value: A}]
        templateRef:
          name: echo-template
          template: echo
      - name: B
        dependencies: [A]
        templateRef:
          name: echo-template
          template: echo
```

```
        arguments:
          parameters: [{name: message, value: B}]
      - name: C
        dependencies: [A]
        templateRef:
          name: echo-template
          template: echo
        arguments:
          parameters: [{name: message, value: C}]
      - name: D
        dependencies: [B, C]
        templateRef:
          name: echo-template
          template: echo
        arguments:
          parameters: [{name: message, value: D}]

EOF
```

To successfully deploy this Workflow we need to temporarily grant admin permissions to argo ServiceAccount with the following command:

```
$ kubectl create rolebinding default-admin --clusterrole=admin
--serviceaccount=argo:default -n argo
```

For a production environment, please follow the [official documentation].

As the next step, create the WorkflowTemplate shown above, and run the following command:

```
kubectl apply -f dag-workflow-template.yaml
```

Then use the following **kubectl create** command to submit the DAG workflow to your Kubernetes cluster with a generated name:

```
kubectl create -f dag-workflow.yaml
```

2. Inspect Workflow Resource

Check whether the WorkflowTemplate and Workflow resources have been created.

Command:

```
$ kubectl -n argo get workflowtemplates.argoproj.io
```

Output:

```
NAME                AGE
```

```
echo-template   13m
```

Command:

```
$ kubectl -n argo get workflow
```

Output:

```
NAME                 STATUS    AGE   MESSAGE
dag-diamond-fnct6    Running   13s
```

You can achieve the same by using the Argo Workflow CLI as follows:

```
$ argo -n argo list
```

Output:

```
NAME                 STATUS      AGE   DURATION   PRIORITY   MESSAGE
dag-diamond-fnct6    Succeeded   34s   30s        0
```

Now let's review the details of the Workflow that we just submitted. The output for the command below will be the same as the information shown when you submitted the Workflow:

```
$ argo -n argo get dag-diamond-fnct6 # Use the name from the previous
command
```

Output:

```
Name:                dag-diamond-fnct6
Namespace:           argo
ServiceAccount:      unset (will run with the default ServiceAccount)
Status:              Succeeded
Conditions:
 PodRunning          False
 Completed           True
Created:             Mon Nov 03 20:39:27 +0000 (52 seconds ago)
Started:             Mon Nov 03 20:39:27 +0000 (52 seconds ago)
Finished:            Mon Nov 03 20:39:57 +0000 (22 seconds ago)
Duration:            30 seconds
Progress:            4/4
ResourcesDuration:   0s*(1 cpu),16s*(100Mi memory)

STEP                  TEMPLATE            PODNAME
DURATION  MESSAGE
 ✔ dag-diamond-fnct6   diamond
 ├─✔ A                 echo-template/echo
dag-diamond-fnct6-echo-2681686565  4s
```

```
 ├─✔ B                  echo-template/echo
dag-diamond-fnct6-echo-2631353708  6s
 ├─✔ C                  echo-template/echo
dag-diamond-fnct6-echo-2648131327  5s
 └─✔ D                  echo-template/echo
dag-diamond-fnct6-echo-2597798470  4s
```

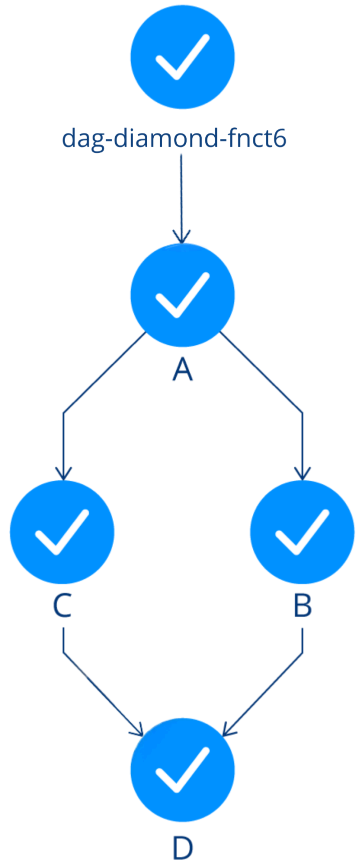
We can also check the logs of the Workflow with the following command:

```
$ argo -n argo logs dag-diamond-fnct6
```

Output:

```
dag-diamond-fnct6-echo-2681686565: A

dag-diamond-fnct6-echo-2681686565: time="2025-11-03T20:39:30.999Z"
level=info msg="sub-process exited" argo=true error="<nil>"

dag-diamond-fnct6-echo-2648131327: C

dag-diamond-fnct6-echo-2648131327: time="2025-11-03T20:39:41.585Z"
level=info msg="sub-process exited" argo=true error="<nil>"

dag-diamond-fnct6-echo-2631353708: B

dag-diamond-fnct6-echo-2631353708: time="2025-11-03T20:39:42.659Z"
level=info msg="sub-process exited" argo=true error="<nil>"

dag-diamond-fnct6-echo-2597798470: D

dag-diamond-fnct6-echo-2597798470: time="2025-11-03T20:39:50.132Z"
level=info msg="sub-process exited" argo=true error="<nil>"
```

As we can see, in this workflow, step A runs first, since it has no dependencies. After A has finished, then steps B and C run in simultaneously. When B and C are completed, step D starts. You can also view the completed steps in Argo UI as shown below.



A simple DAG workflow

3. Clean Up the Resources

We successfully created an argo Workflow. To keep our working cluster nice and clean, we are going to clean up the resources we created using the commands below:

```
$ kubectl -n argo delete workflow dag-diamond-fnct6
$ kubectl -n argo delete workflowtemplates.argoproj.io echo-template
```

4. Summary

In this exercise, we successfully implemented a lab setup by deploying DAG Workflows. Within a Kubernetes environment, we demonstrated how to define dependencies between tasks which can be well-suited for use cases that involve complex dependencies and workflows. For example, data extraction must be completed before data transformation, and transformation must finish before data loading. Argo Workflow DAG allows you to model and execute these dependencies efficiently.

Lab 4.3. CI/CD Using Argo Workflows

This lab will give you an overview of implementing a CI/CD pipeline using Argo Workflow. Imagine you have a microservices-based application deployed on a Kubernetes cluster. You want to implement a CI/CD pipeline using Argo Workflows to automate the build, test, and deployment process for each microservice.

Objective

- Define workflows that include various steps such as building, testing, and deploying applications.
- Incorporate testing steps into your workflows to ensure the quality of your applications.
- Run unit tests, integration tests, and other relevant tests as part of the CI/CD process.

Prerequisites

- Kubernetes cluster with argo-workflow controller.
  - See Chapter 2's "Deploying Kubernetes for Argo" section for details on how to set up a cluster
- Argo Workflows CLI (optional).

Create a CI/CD Pipeline with Argo Workflow

1. Create Argo Workflow

We can start by creating and managing our workflows as code. We will create a Python application workflow with the following steps:

- Build: The build step builds the image with the latest changes, using a Python 3 image for this scenario.

- Tests: The test step mounts a volume with test files and runs unit tests with the Python unittest library.
- Deployment: The deploy step runs the Python container and prints deploy. Normally, this step would involve pushing the tested code to a container registry, like AWS ECR or Harbor, and then deploying it to the production environment.

This lab provides a basic setup for Argo Workflow CI/CD. Depending on your specific requirements, you may want to enhance the workflow with additional steps, such as testing, linting, or pushing the Docker image to a registry.

Define the Argo Workflow manifest (`workflow-ci.yaml`) that describes the sequence of tasks and dependencies.

```
cat <<EOF > workflow-ci.yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  name: python-app
spec:
  entrypoint: python-app
  templates:
  - name: python-app
    steps:
    - - name: build
        template: build
    - - name: test
        template: test
    - - name: deploy
        template: deploy
  - name: build
    container:
      image: python:3.11
      command: [python]
      args: ["-c", "print('build')"]
  - name: test
    container:
      image: python:3.11
      command: [python]
      args: ["-m", "unittest", "discover", "-s", "/app/tests"]
      volumeMounts:
      - name: test-volume
        mountPath: /app/tests
    volumes:
    - name: test-volume
```

```
      hostPath:
        path: /path/to/tests
  - name: deploy
    container:
      image: python:3.11
      command: [python]
      args: ["-c", "print('deploy')"]
EOF
```

2. Deploy the Workflow

Deploy the workflow by running the following command:

```
$ kubectl -n argo apply -f workflow-ci.yaml
```

This will create an Argo Workflow with the name **`python-app`** along with their respective templates and steps.

3. Inspect the Workflow

Now, let's execute the **`argo get`** command to retrieve information about the workflow, including its status:

```
$ argo -n argo get python-app
```

Output:

```
Name:                   python-app
Namespace:              argo-workflows
ServiceAccount:         unset (will run with the default ServiceAccount)
Status:                 Succeeded
Conditions:
 PodRunning             False
 Completed              True
Created:                Mon Jan 08 00:07:51 +0100 (21 minutes ago)
Started:                Mon Jan 08 00:07:51 +0100 (21 minutes ago)
Finished:               Mon Jan 08 00:08:39 +0100 (20 minutes ago)
Duration:               48 seconds
Progress:               3/3
ResourcesDuration:      26s*(1 cpu),16s*(100Mi memory)

STEP           TEMPLATE    PODNAME                          DURATION
MESSAGE
```

```
✔ python-app  python-app
├───✔ build   build       python-app-build-3831382643  19s
├───✔ test    test        python-app-test-461837862    4s
└───✔ deploy  deploy      python-app-deploy-988129288  5s
```

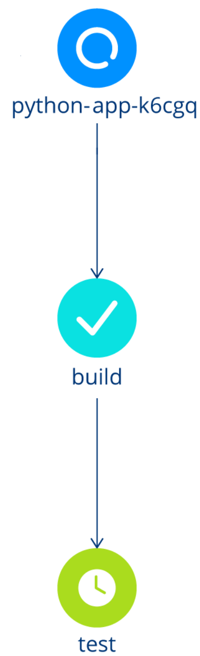
The work can be observed in the cluster as a set of pods (one for each step). Run the following command:

```
$ kubectl get pods -n argo
```

Output:

```
NAME                                    READY   STATUS      RESTARTS
AGE
argo-server-66c9577ddc-cf9zf            1/1     Running     0
44m
python-app-build-3831382643             0/2     Completed   0
70s
python-app-deploy-988129288             0/2     Completed   0
25s
python-app-test-461837862               0/2     Completed   0
35s
workflow-controller-5bb95ffc45-pvp9d    1/1     Running     0
48m
```

You can also see the Workflow progress in Argo UI:



CI/CD Workflow progress overview

We can also view the logs:

```
$ argo -n argo logs python-app
```

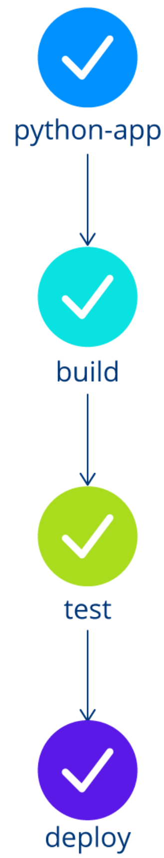
Output:

```
python-app-build-3831382643: build

python-app-build-3831382643: time="2025-11-03T21:05:42.251Z"
level=info msg="sub-process exited" argo=true error="<nil>"

python-app-test-461837862:

python-app-test-461837862:
----------------------------------------------------------------------

python-app-test-461837862: Ran 0 tests in 0.000s

python-app-test-461837862:

python-app-test-461837862: OK

python-app-test-461837862: time="2025-11-03T21:05:54.210Z" level=info
msg="sub-process exited" argo=true error="<nil>"

python-app-deploy-988129288: deploy
```

```
python-app-deploy-988129288: time="2025-11-03T21:06:04.315Z"
level=info msg="sub-process exited" argo=true error="<nil>"
```

Refresh the UI of Argo Workflows. You should see a new workflow that is processing.

Refresh again after a few minutes and you should see the following screen with a completed workflow.



CI/CD Workflow overview

4. Summary

In this chapter, we completed a lab setup by deploying a CI/CD pipeline using Argo Workflow to show how to establish a robust and automated pipeline for building, testing, and deploying applications in Kubernetes, promoting a DevOps culture and enhancing software delivery practices. Here we mention a few benefits of such an implementation:

- Automation and efficiency: Argo Workflows automates the CI/CD pipeline, reducing manual intervention and enhancing efficiency.
- Consistency and reproducibility: With Argo Workflows, CI/CD processes are defined declaratively, ensuring consistency across multiple runs and environments.

- Scalability: Argo Workflows provides scalability, allowing CI/CD processes to scale with the growing needs of the application.
- Visibility and monitoring: Argo Workflows' UI and CLI provide visibility into workflow execution, allowing teams to monitor progress and troubleshoot issues.

Flexibility and customization: Argo Workflows offers flexibility in defining custom workflows, allowing teams to tailor CI/CD pipelines to their specific requirements.

Lab 5.1. Installing Argo Rollouts

Objective

Set up a local Kubernetes cluster using Kind, install Argo Rollouts, and understand how to access Argo Rollout resources.

Prerequisites

- Basic understanding of Docker, Kubernetes, and command-line interface operations.
- Access to a computer with an internet connection.
- An installation of Kubernetes that you have full control over
  - See Chapter 2's Deploying Kubernetes for Argo section for details on how to set one up for yourself

Install Cluster and Argo Rollouts

NOTE*: Steps 1-4 might not be necessary if you already followed the setup during a previous chapter.*

1. Installing Docker

Ensure Docker is installed and running on your machine.

- Installation instructions can be found on the Docker website.

2. Installing Kind

Download and install Kind following the instructions from the Kind official website.

3. Creating a Kubernetes Cluster with Kind

Create a cluster by running the following command:

```
kind create cluster
```

This command creates a single-node Kubernetes cluster running inside a Docker container named **`kind-kind`**.

4. Installing kubectl

Instructions for downloading kubectl can be found in the Kubernetes official documentation.

5. Deploy Argo Rollouts

Create a namespace for Argo Rollouts using the following command:

```
kubectl create namespace argo-rollouts
```

Deploy Argo Rollouts using the quick start manifest:

```
kubectl apply -n argo-rollouts -f
https://github.com/argoproj/argo-rollouts/releases/download/v1.8.3/install.yaml
```

This will install custom resource definitions as well as the Argo Rollouts controller.

During this course we use Argo Rollouts in version 1.8.3. We recommend using the same version to ensure consistent results.

Verify that Argo Rollouts is installed by running the following command:

```
kubectl get pods -n argo-rollouts
```

6. Install Rollouts kubectl Plugin

Unlike Argo CD and Argo Workflows, Argo Rollouts uses a kubectl plugin as its CLI client.

Download the latest Argo Rollouts kubectl plugin version from https://github.com/argoproj/argo-rollouts/releases/latest/.

On Ubuntu 24.04, you can install the CLI using the following commands:

```
$ wget
https://github.com/argoproj/argo-rollouts/releases/download/v1.8.3/kub
ectl-argo-rollouts-linux-amd64 -O kubectl-argo-rollouts
```

```
$ chmod +x kubectl-argo-rollouts
```

```
$ sudo mv kubectl-argo-rollouts /usr/local/bin/
```

More detailed installation instructions can be found via the [CLI installation documentation](https://argoproj.github.io/argo-rollouts/installation/#kubectl-plugin-installation).

This is also available in Mac, Linux and WSL Homebrew. Use the following command:

```
brew install argoproj/tap/kubectl-argo-rollouts
```

Verify that the **argo** CLI is installed correctly by running the following command:

```
kubectl argo rollouts version
```

7. UI Dashboard

For the sake of completeness it needs to be mentioned that Argo Rollouts ships with a fully fledged UI Dashboard. It can be accessed via the **kubectl argo rollouts dashboard** command and provides a nice overview and basic commands for administration.

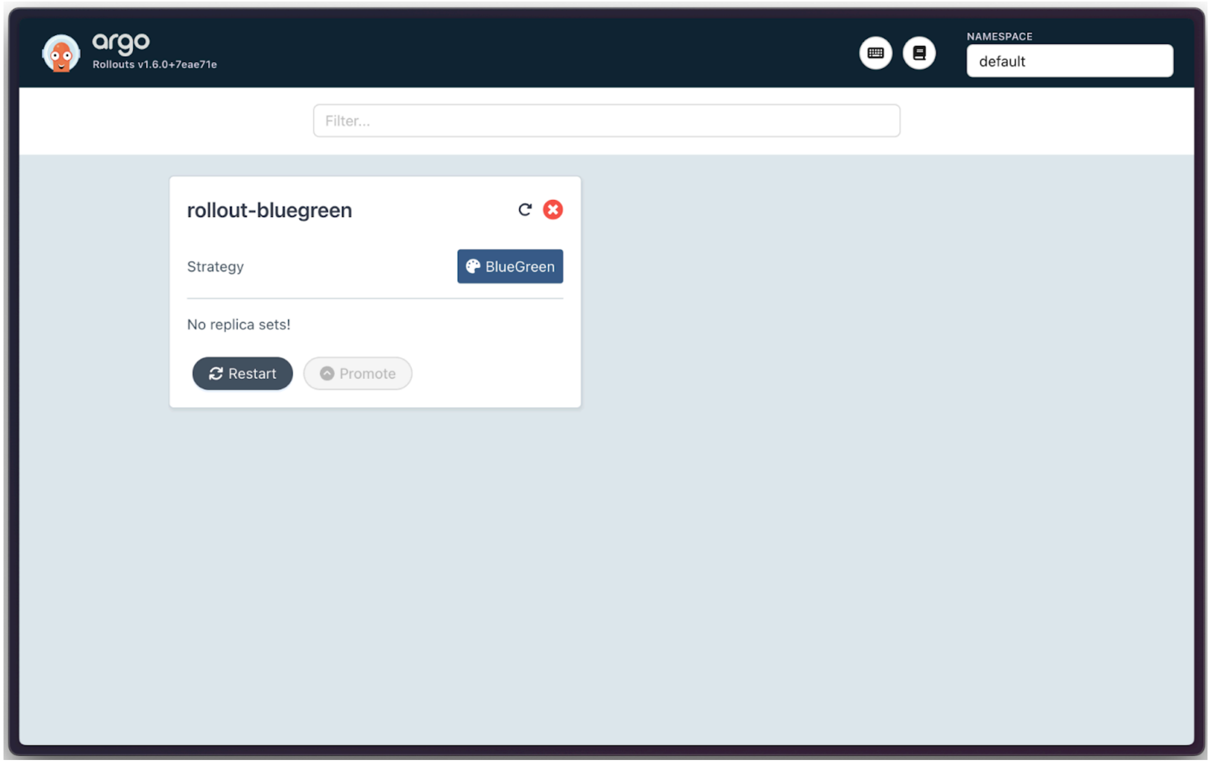
```
$ kubectl argo rollouts dashboard
```

Output:

```
INFO[0000] Argo Rollouts Dashboard is now available at
http://localhost:3100/rollouts

INFO[0000] [core] [Channel #1 SubChannel #2]grpc:
addrConn.createTransport failed to connect to {Addr: "0.0.0.0:3100",
ServerName: "0.0.0.0:3100", }. Err: connection error: desc =
"transport: Error while dialing: dial tcp 0.0.0.0:3100: connect:
connection refused"
```

Despite any “connection refused” errors, you can now access it via the UI at **http://localhost:3100** or your VM’s public IP address at port 3100.

As the Dashboard is self-explanatory, we will not discuss it in detail during this course.



The Argo Rollouts Dashboard displaying a sample app "rollout-bluegreen"

NOTE*: If no rollout resources are in place, the dashboard will display "Loading…".*

8. Optional: Shell Auto-Completion

To get easy access to Argo Rollout resources, the CLI can add shell completion code for several shells. For bash, you can use the following command:

```
source <(kubectl-argo-rollouts completion bash)
```

Other shells are supported as well. Please refer to the [completion command documentation]() for more details.

9. Using Argo Rollouts

There are a wide variety of commands that you can use to control Argo Rollouts via the CLI, as described in the **`-h`** output for the **`kubectl argo rollouts`** command. As a kubectl plugin, it uses the Kubernetes API to perform all management tasks.

Here is a list of the most common commands to operate with Argo Rollouts:

```
kubectl get rollout
```

```
kubectl argo rollouts get rollout
```

```
kubectl argo rollouts promote
```

```
kubectl argo rollouts undo
```

Lab 5.2. Argo Rollouts Blue-Green

Let's dig into it by creating a blue-green deployment scenario. It enables us to verify a version upgrade before the live traffic hits our service. It is easy to understand and therefore one of the most commonly used ways to roll out new versions of software without any downtime.

Objective

This lab aims to give a reader an idea of the "look and feel" of Argo Rollouts. It will demonstrate how to realize a simple blue-green scenario with Argo Rollouts. As blue-green is the most basic deployment pattern that rollout supports, this is a great introduction to the fundamental functionality of Argo Rollouts.

Prerequisites

- Kubernetes cluster with the argo-rollouts controller
- kubectl with the argo-rollouts plugin (optional)

Creating Blue-Green Deployments with Argo Rollouts

1. Install Resources

For the beginning, let's check for existing rollouts using this command:

```
$ kubectl get rollout
```

Output:

```
No resources found in default namespace.
```

As expected, there are no rollouts (yet) to be found in our cluster. Let's create one with the following command:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: rollout-bluegreen
spec:
  replicas: 2
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: rollout-bluegreen
  template:
    metadata:
      labels:
        app: rollout-bluegreen
    spec:
      containers:
      - name: rollouts-demo
        image: argoproj/rollouts-demo:blue
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
  strategy:
    blueGreen:
      activeService: rollout-bluegreen-active
      previewService: rollout-bluegreen-preview
      autoPromotionEnabled: false
EOF
```

Output:

```
rollout.argoproj.io/rollout-bluegreen created
```

Check whether the Rollout resource has been created running the command below:

```
$ kubectl get rollout
```

Output:

```
NAME                 DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
rollout-bluegreen    2                                            75s
```

It has been created, but is not ready yet. Let’s explore the reasons behind using the Argo Rollouts kubectl plugin to see if we can better understand its functionality.

The plugin enables us to get status information of a specific rollout and can be queried in the form: **`kubectl argo rollouts get ro <rollout-name>`**.

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:              rollout-bluegreen

Namespace:         default

Status:            ✖ Degraded

Message:           InvalidSpec: The Rollout "rollout-bluegreen" is
invalid: spec.strategy.blueGreen.activeService: Invalid value:
"rollout-bluegreen-active": service "rollout-bluegreen-active" not
found

Strategy:          BlueGreen

Replicas:

  Desired:         2

  Current:         0

  Updated:         0

  Ready:           0

  Available:       0


NAME                 KIND     STATUS      AGE  INFO

⟳ rollout-bluegreen  Rollout  ✖ Degraded  9s
```

The resource status is degraded, as not all requirements are met: If we look closely in our rollout manifest we see that we defined two services **`activeService`** and **`previewService`**. We need to make sure that the named services are available.

Let's create them with the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
```

```
metadata:
  creationTimestamp: null
  labels:
    app: rollout-bluegreen-active
  name: rollout-bluegreen-active
spec:
  ports:
  - name: "80"
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: rollout-bluegreen
  type: ClusterIP
status:
  loadBalancer: {}
EOF
```

As mentioned, the rollouts resource references a second service. A so-called “preview” service. A preview service enables a preview stack to be reachable by an administrator. It does so without serving public traffic.

If we want the preview to go live, we need to "promote" the rollout. "Promotion" refers to setting a service live.

Therefore, we will create a preview service resource to be able to check our application before promotion (aka setting it live). Run the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: rollout-bluegreen-preview
  name: rollout-bluegreen-preview
spec:
  ports:
  - name: "80"
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: rollout-bluegreen
  type: ClusterIP
```

```
status:
  loadBalancer: {}
EOF
```

If we now check our rollout, we will eventually see a **`Healthy`** status. Run the command below:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:            rollout-bluegreen
Namespace:       default
Status:          ✔ Healthy
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:blue (stable, active)
Replicas:
  Desired:       2
  Current:       2
  Updated:       2
  Ready:         2
  Available:     2

NAME                                          KIND        STATUS
AGE  INFO
⟳ rollout-bluegreen                           Rollout     ✔ Healthy
50s
└──# revision:1
   └──⧉ rollout-bluegreen-5ffd47b8d4           ReplicaSet  ✔ Healthy
14s  stable,active
      ├──□ rollout-bluegreen-5ffd47b8d4-mqc25  Pod         ✔ Running
4s   ready:1/1
      └──□ rollout-bluegreen-5ffd47b8d4-q4bgf  Pod         ✔ Running
3s   ready:1/1
```

2. Perform an Upgrade

Now that we have a running application, let's try to perform a version upgrade using the blue-green method. Therefore, we'll adjust our image to deploy **`argoproj/rollouts-demo:green`** instead of **`blue`**. Run the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: rollout-bluegreen
```

```
spec:
  replicas: 2
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: rollout-bluegreen
  template:
    metadata:
      labels:
        app: rollout-bluegreen
    spec:
      containers:
      - name: rollouts-demo
        image: argoproj/rollouts-demo:green
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
  strategy:
    blueGreen:
      activeService: rollout-bluegreen-active
      previewService: rollout-bluegreen-preview
      autoPromotionEnabled: false
EOF
```

The Rollout status moves from “**`Healthy`**” to “**`Paused`**”, indicating that a rollout is in progress and waits for further action.

Please note, that we explicitly set **`autoPromotionEnabled`** to **`false`** - we can skip the pausing phase and directly promote by setting this value to **`true`**. Run the command below:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:             rollout-bluegreen
Namespace:        default
Status:           ‖ Paused
Message:          BlueGreenPause
Strategy:         BlueGreen
Images:           argoproj/rollouts-demo:blue (stable, active)
                  argoproj/rollouts-demo:green (preview)
Replicas:
  Desired:        2
  Current:        4
  Updated:        2
```

```
  Ready:          2
  Available:      2

NAME                                             KIND         STATUS
AGE   INFO
⟳ rollout-bluegreen                              Rollout      ॥ Paused
91s
├──# revision:2
│  └──⧉ rollout-bluegreen-75695867f              ReplicaSet  ✔ Healthy
6s    preview
│     ├──□ rollout-bluegreen-75695867f-m6pxh    Pod          ✔ Running
6s    ready:1/1
│     └──□ rollout-bluegreen-75695867f-nr2rh    Pod          ✔ Running
6s    ready:1/1
└──# revision:1
   └──⧉ rollout-bluegreen-5ffd47b8d4             ReplicaSet  ✔ Healthy
55s   stable,active
      ├──□ rollout-bluegreen-5ffd47b8d4-mqc25   Pod          ✔ Running
45s   ready:1/1
      └──□ rollout-bluegreen-5ffd47b8d4-q4bgf   Pod          ✔ Running
44s   ready:1/1
```

Let's investigate the rollout a little further and check replicasets with the command below:

```
$ kubectl get replicaset
```

Output:

```
NAME                           DESIRED   CURRENT   READY   AGE
rollout-bluegreen-5ffd47b8d4   2         2         2       80s
rollout-bluegreen-75695867f    2         2         2       31s
```

Argo rollout created a second replicaset, which is used to manage the different pod versions. Lets promote the new version.

Command:

```
$ kubectl argo rollouts promote rollout-bluegreen
```

Output:

```
rollout 'rollout-bluegreen' promoted
```

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:            rollout-bluegreen
Namespace:       default
Status:          ✔ Healthy
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:green (stable, active)
Replicas:
  Desired:       2
  Current:       2
  Updated:       2
  Ready:         2
  Available:     2

NAME                                           KIND        STATUS
AGE    INFO
⟳ rollout-bluegreen                            Rollout     ✔ Healthy
2m40s
├──# revision:2
│  └──⧉ rollout-bluegreen-75695867f             ReplicaSet  ✔ Healthy
75s    stable,active
│     ├──□ rollout-bluegreen-75695867f-m6pxh   Pod         ✔ Running
75s    ready:1/1
│     └──□ rollout-bluegreen-75695867f-nr2rh   Pod         ✔ Running
75s    ready:1/1
└──# revision:1
   └──⧉ rollout-bluegreen-5ffd47b8d4            ReplicaSet  •
ScaledDown   2m4s
      ├──□ rollout-bluegreen-5ffd47b8d4-mqc25  Pod         ◌
Terminating  114s    ready:1/1
      └──□ rollout-bluegreen-5ffd47b8d4-q4bgf  Pod         ◌
Terminating  113s    ready:1/1
```

Our new revision changed from "**`preview`**" to "**`stable,active`**" - indicating that the new revision is live.

You may also see that the first revision will display "**`delay`**" followed by a counter. Eventually, it will go into a "**`ScaledDown`**" status.

We can even see this by checking our service with the command below:

```
$ kubectl describe svc rollout-bluegreen-active
```

Output:

```
Name:                     rollout-bluegreen-active
Namespace:                default
Labels:                   app=rollout-bluegreen-active
Annotations:
argo-rollouts.argoproj.io/managed-by-rollouts: rollout-bluegreen
Selector:
app=rollout-bluegreen,rollouts-pod-template-hash=75695867f
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.96.227.100
IPs:                      10.96.227.100
Port:                     80  80/TCP
TargetPort:               80/TCP
Endpoints:                10.244.0.43:80,10.244.0.44:80
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
```

Note that the Selector **`rollouts-pod-template-hash`** has the same value as the new ReplicaSet.

We just successfully performed a deployment using blue-green methodology.

3. Perform a Rollback

Let's assume we want to roll back from the new green to the old blue image.

Command:

```
$ kubectl argo rollouts undo rollout-bluegreen
```

Output:

```
rollout 'rollout-bluegreen' undo
```

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:            rollout-bluegreen
Namespace:       default
Status:          ॥ Paused
Message:         BlueGreenPause
Strategy:        BlueGreen
```

```
Images:           argoproj/rollouts-demo:blue (preview)
                  argoproj/rollouts-demo:green (stable, active)
Replicas:
  Desired:        2
  Current:        4
  Updated:        2
  Ready:          2
  Available:      2

NAME                                             KIND         STATUS
AGE     INFO
⟳ rollout-bluegreen                              Rollout      ‖ Paused
3m52s
├──# revision:3
│  └──⧉ rollout-bluegreen-5ffd47b8d4              ReplicaSet  ✔ Healthy
3m16s  preview
│     ├──□ rollout-bluegreen-5ffd47b8d4-2lvcq    Pod          ✔ Running
3s      ready:1/1
│     └──□ rollout-bluegreen-5ffd47b8d4-k8sqf    Pod          ✔ Running
3s      ready:1/1
└──# revision:2
   └──⧉ rollout-bluegreen-75695867f               ReplicaSet  ✔ Healthy
2m27s  stable,active
      ├──□ rollout-bluegreen-75695867f-m6pxh     Pod          ✔ Running
2m27s  ready:1/1
      └──□ rollout-bluegreen-75695867f-nr2rh     Pod          ✔ Running
2m27s  ready:1/1
```

Note, that “undo” alone did not set the blue image active. The rollout is now again in the pausing phase, waiting for promotion of the rollout. Run the following command:

```
$ kubectl argo rollouts promote rollout-bluegreen
```

Output:

```
rollout 'rollout-bluegreen' promoted
```

Checking our rollout and active service once again, we see, that the selector changed back to our old ReplicaSet.

Command:

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

Output:

```
Name:            rollout-bluegreen
Namespace:       default
Status:          ✔ Healthy
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:blue (stable, active)
                 argoproj/rollouts-demo:green
Replicas:
  Desired:       2
  Current:       4
  Updated:       2
  Ready:         2
  Available:     2

NAME                                        KIND        STATUS
AGE    INFO
⟳ rollout-bluegreen                         Rollout     ✔ Healthy
4m17s
├──# revision:3
│  └──⧉ rollout-bluegreen-5ffd47b8d4         ReplicaSet  ✔ Healthy
3m41s  stable,active
│     ├──□ rollout-bluegreen-5ffd47b8d4-2lvcq  Pod         ✔ Running
28s    ready:1/1
│     └──□ rollout-bluegreen-5ffd47b8d4-k8sqf  Pod         ✔ Running
28s    ready:1/1
└──# revision:2
   └──⧉ rollout-bluegreen-75695867f          ReplicaSet  ✔ Healthy
2m52s  delay:23s
      ├──□ rollout-bluegreen-75695867f-m6pxh   Pod         ✔ Running
2m52s  ready:1/1
      └──□ rollout-bluegreen-75695867f-nr2rh   Pod         ✔ Running
2m52s  ready:1/1
```

Command:

```
$ kubectl describe svc rollout-bluegreen-active
```

Output:

```
Name:                      rollout-bluegreen-active
Namespace:                 default
Labels:                    app=rollout-bluegreen-active
Annotations:
argo-rollouts.argoproj.io/managed-by-rollouts: rollout-bluegreen
Selector:
app=rollout-bluegreen,rollouts-pod-template-hash=5ffd47b8d4
```

```
Type:                      ClusterIP
IP Family Policy:          SingleStack
IP Families:               IPv4
IP:                        10.96.227.100
IPs:                       10.96.227.100
Port:                      80  80/TCP
TargetPort:                80/TCP
Endpoints:                 10.244.0.45:80,10.244.0.46:80
Session Affinity:          None
Internal Traffic Policy:   Cluster
Events:                    <none>
```

4. Clean Up Resources

We successfully used the blue-green deployment pattern to deploy an application and even performed a rollback. To keep our working cluster nice and clean, we are going to clean up resources we created.

Command:

```
$ kubectl delete rollout rollout-bluegreen
```

Output:

```
rollout.argoproj.io "rollout-bluegreen" deleted
```

Command:

```
$ kubectl delete svc rollout-bluegreen-active
```

Output:

```
rollout-bluegreen-preview
service "rollout-bluegreen-active" deleted
service "rollout-bluegreen-preview" deleted
```

Lab 5.3. Migrating an Existing Deployment to Argo Rollouts

Chances are, that you are not starting with a fresh Kubernetes installation but already have a running cluster with deployed workloads. Argo Rollouts has this scenario in mind and provides a migration path to migrate Deployments to Rollout resources.

Objective

Migrate a vanilla Kubernetes Deployment to an Argo Rollout resource.

Prerequisites

- Kubernetes cluster with an argo-rollouts controller.
- kubectl with an argo-rollouts plugin (optional).

Transitioning to Argo Rollouts

1. Preparing resources

For this lab, we will create an NGINX deployment—a task you may have already undertaken numerous times. Run the command below:

```
$ kubectl create deploy nginx-deployment --image=nginx --replicas=3
```

Now check our running pods and deployments using the following command:

```
kubectl get po,deploy
```

Output:

```
NAME                                     READY   STATUS    RESTARTS
AGE
pod/nginx-deployment-7457467ffd-8r8jq   1/1     Running   0
9s
pod/nginx-deployment-7457467ffd-s79v2   1/1     Running   0
9s
pod/nginx-deployment-7457467ffd-s9b4h   1/1     Running   0
9s

NAME                                READY   UP-TO-DATE   AVAILABLE
AGE
deployment.apps/nginx-deployment   3/3     3            3           9s
```

2. Convert Deployment to Rollout

Now we want to use the deployment definition to reference it in a new rollout. Run the command below:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: nginx-rollout
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deployment
  workloadRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {duration: 10s}

EOF
```

Note the field **`workloadRef`**, which references the **`nginx-deployment`** resource.

As a result, we have 6 nginx instances running, 3 managed by our vanilla deployment, 3 by the newly created rollout. Run the command below:

```
kubectl get ro,deploy,po
```

Output:

```
NAME                                  DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
rollout.argoproj.io/nginx-rollout   3         3         3            3
4s

NAME                                 READY   UP-TO-DATE   AVAILABLE
AGE
deployment.apps/nginx-deployment   3/3     3            3
2m59s

NAME                                       READY   STATUS    RESTARTS
AGE
pod/nginx-deployment-7457467ffd-8r8jq   1/1     Running   0
2m59s
pod/nginx-deployment-7457467ffd-s79v2   1/1     Running   0
2m59s
pod/nginx-deployment-7457467ffd-s9b4h   1/1     Running   0
2m59s
pod/nginx-rollout-6d7df6cfcb-4snrs      1/1     Running   0
4s
pod/nginx-rollout-6d7df6cfcb-bm46n      1/1     Running   0
4s
pod/nginx-rollout-6d7df6cfcb-jg7p8      1/1     Running   0
4s
```

3. Scale Down Deployment

To finish the migration, we now need to manually scale down the deployment. Run the following command:

```
$ kubectl scale deployment/nginx-deployment --replicas=0
```

Output:

```
NAME                                  DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
rollout.argoproj.io/nginx-rollout   3         3         3            3
9m58s

NAME                                 READY   UP-TO-DATE   AVAILABLE
AGE
```

```
deployment.apps/nginx-deployment   0/0     0            0
12m

NAME                                READY   STATUS    RESTARTS   AGE
pod/nginx-rollout-6d7df6cfcb-4snrs  1/1     Running   0
9m58s
pod/nginx-rollout-6d7df6cfcb-bm46n  1/1     Running   0
9m58s
pod/nginx-rollout-6d7df6cfcb-jg7p8  1/1     Running   0
9m58s
```

This leaves you with an up-and-running workload, managed by a rollout resource!

The step of scaling down the deployment once referenced by the Rollout resource can be taken over by the Argo Rollout controller. A special **`scaleDown`** parameter exists that enables administrators to specify how the deployment should be scaled down (**`never`**, **`onsuccess`**, **`progressively`**).

After confirming the deployment is scaled down, scale it up one more time with the following command:

```
$ kubectl scale deployment/nginx-deployment --replicas=3
```

And apply the following new rollout spec which includes the scaleDown parameter:

```
$ cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: nginx-rollout
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deployment
  workloadRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
    scaleDown: onsuccess
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {duration: 10s}
```

```
EOF
```

This will provide the same result as before, except this time there was no need for any manual intervention!

Command:

```
~$ kubectl get ro,deploy,po
```

Output:

```
NAME                                DESIRED   CURRENT   UP-TO-DATE   
AVAILABLE   AGE
rollout.argoproj.io/nginx-rollout   3         3         3            3
12m

NAME                               READY   UP-TO-DATE   AVAILABLE   
AGE
deployment.apps/nginx-deployment   0/0     0            0
15m

NAME                                 READY   STATUS    RESTARTS   AGE
pod/nginx-rollout-6d7df6cfcb-4snrs   1/1     Running   0          12m
pod/nginx-rollout-6d7df6cfcb-bm46n   1/1     Running   0          12m
pod/nginx-rollout-6d7df6cfcb-jg7p8   1/1     Running   0          12m
```

More details on the feature can be found on the Rollout Migration documentation page.

4. Clean Up Resources

Make sure to leave the cluster nice and clean.

Command:

```
$ kubectl delete rollout nginx-rollout
```

Output:

```
rollout.argoproj.io "nginx-rollout" deleted
```

Command:

```
$ kubectl delete deployment nginx-deployment
```

Output:

```
deployment.apps "nginx-deployment" deleted
```

Lab 6.1. Setting Up Event Triggers with Argo

In this chapter, we begin our hands-on journey with Argo Events, starting from setting up its basic components and triggering simple workflows with HTTP requests. After a theoretical overview, the first lab focuses on practical implementation, allowing users to familiarize themselves with Argo Events by using simple curl commands to initiate workflows. Building on this foundational knowledge, the chapter advances to a lab that integrates Argo Events with an external system, Apache Pulsar. This step-by-step exploration not only enhances understanding of Argo Events in real-world scenarios but also demonstrates its capability to seamlessly integrate with other technologies, showcasing its versatility in managing event-driven architectures in cloud-native environments.

Objective

- Set up a laboratory environment that demonstrates the integration and functionality of Argo Events and Argo Workflows.
- Install and configure the necessary components to trigger a workflow through an event.

Prerequisites

- A Kubernetes cluster is available.
- kubectl is installed and configured to communicate with your Kubernetes cluster.
- Internet access for downloading necessary files and packages.

Install and Use Argo Events

1. Installing Argo Workflows

Because we trigger a workflow we need to install Argo Workflows first. For that create the respective namespace **`argo`** and apply the installation YAML to the new namespace. Run the commands below:

```
kubectl create namespace argo
```

```
kubectl apply -n argo -f
https://github.com/argoproj/argo-workflows/releases/download/v3.7.3/install.yaml
```

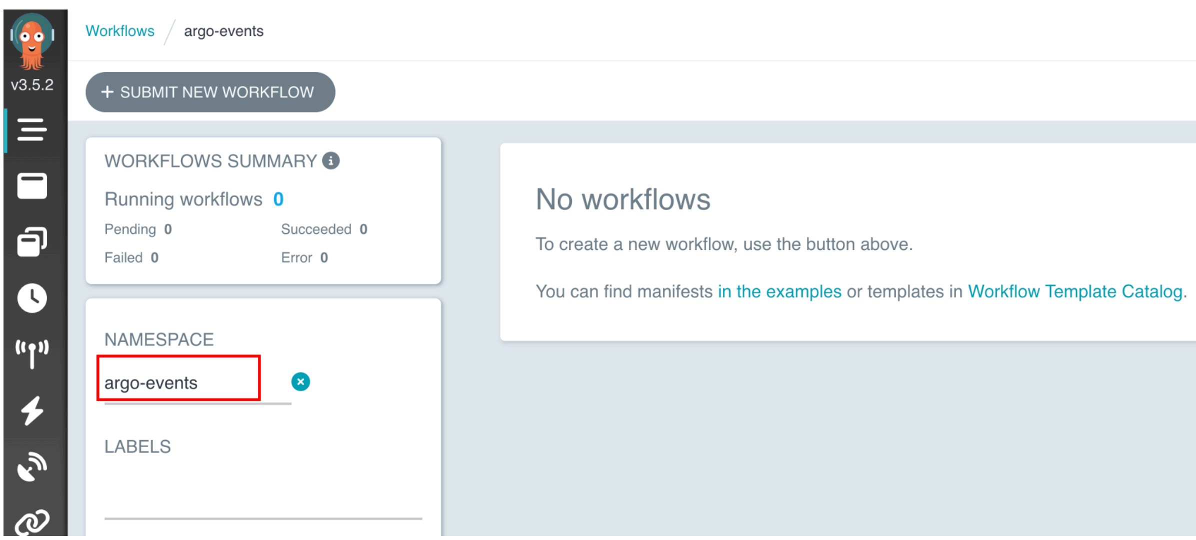
In the next step, we want to access Argo Workflows through the UI. For that we need to change the authentication mode of the Argo Server to "**`server`**". It ensures that the server can properly authenticate requests. Run the command below:

```
kubectl patch deployment argo-server --namespace argo --type='json'
-p='[{"op": "replace", "path":
"/spec/template/spec/containers/0/args", "value": ["server",
"--auth-mode=server"]}]'
```

Lastly, port forward the Argo server deployment to access it on https://localhost:2746 locally using the following command:

```
kubectl -n argo port-forward deployment/argo-server 2746:2746
```

You should see the following screen:



Start screen of Argo Workflows

Make sure to select "**`argo-events`**" as the namespace.

2. Install Argo Events and the Needed Components

Creates a new namespace named '**`argo-events`**', which is used to separate and manage the resources related to Argo Events. After that, install Argo Events in your cluster by applying the installation manifest from the given URL. Run the following commands:

```
kubectl create namespace argo-events
```

```
kubectl apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/manifest
s/install.yaml
```

The next command applies a validating webhook for Argo Events. Validating webhooks are used to ensure that incoming requests to the Kubernetes API server are valid:

```
kubectl apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/manifest
s/install-validating-webhook.yaml
```

For setting up a native EventBus in the **argo-events** namespace, which handles event transportation in Argo Events, apply the configuration with this command:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/eventbus/native.yaml
```

Next we need to define an EventSource configuration that listens for webhook events in Argo Events, apply the following configuration using this command:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/event-sources/webhook.yaml
```

For the Sensor to properly interact with Kubernetes resources, apply the necessary RBAC policies with the command below:

```
kubectl apply -n argo-events -f
https://raw.githubusercontent.com/argoproj/argo-events/master/examples
/rbac/sensor-rbac.yaml
```

Similarly, apply RBAC policies for Workflows to ensure they have the necessary permissions in Kubernetes. Run the following command:

```
kubectl apply -n argo-events -f
https://raw.githubusercontent.com/argoproj/argo-events/master/examples
/rbac/workflow-rbac.yaml
```

Set up a Sensor to trigger workflows based on webhook events by creating this Sensor configuration in a file called **webhook.yaml**. Run the following command:

```
cat <<EOF | kubectl -n argo-events apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Sensor
```

```
metadata:
  name: webhook
spec:
  template:
    serviceAccountName: operate-workflow-sa
  dependencies:
    - name: test-dep
      eventSourceName: webhook
      eventName: example
  triggers:
    - template:
        name: webhook-workflow-trigger
        k8s:
          operation: create
          source:
            resource:
              apiVersion: argoproj.io/v1alpha1
              kind: Workflow
              metadata:
                generateName: webhook-
              spec:
                entrypoint: cowsay
                arguments:
                  parameters:
                  - name: message
                    # the value will get overridden by event payload
from test-dep
                    value: hello world
                templates:
                - name: cowsay
                  inputs:
                    parameters:
                    - name: message
                  container:
                    image: rancher/cowsay:latest
                    command: [cowsay]
                    args: ["{{inputs.parameters.message}}"]
          parameters:
            - src:
                dependencyName: test-dep
                dataKey: body
              dest: spec.arguments.parameters.0.value
EOF
```

Expose the **event-source** pod via port forwarding to consume requests over HTTP:

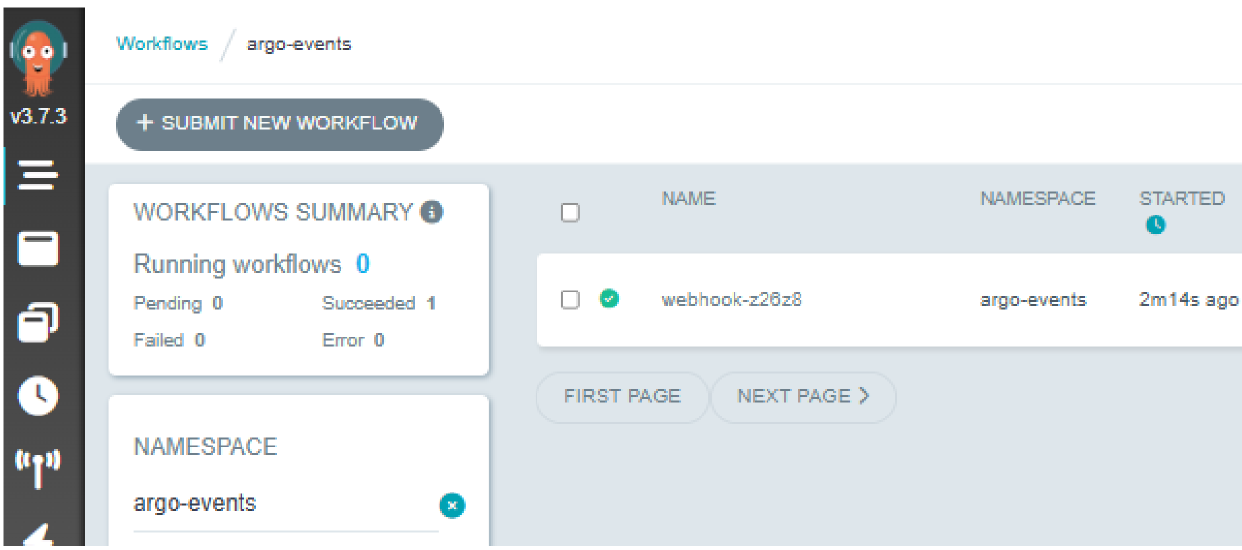
```
kubectl -n argo-events port-forward $(kubectl -n argo-events get pod
-l eventsource-name=webhook -o name) 12000:12000 &
```

Finally, simulate an external event that triggers the workflow. Send a test webhook event to the Event Source with this **curl** command:

```
curl -d '{"message":"this is my first webhook"}' -H "Content-Type:
application/json" -X POST http://localhost:12000/example
```

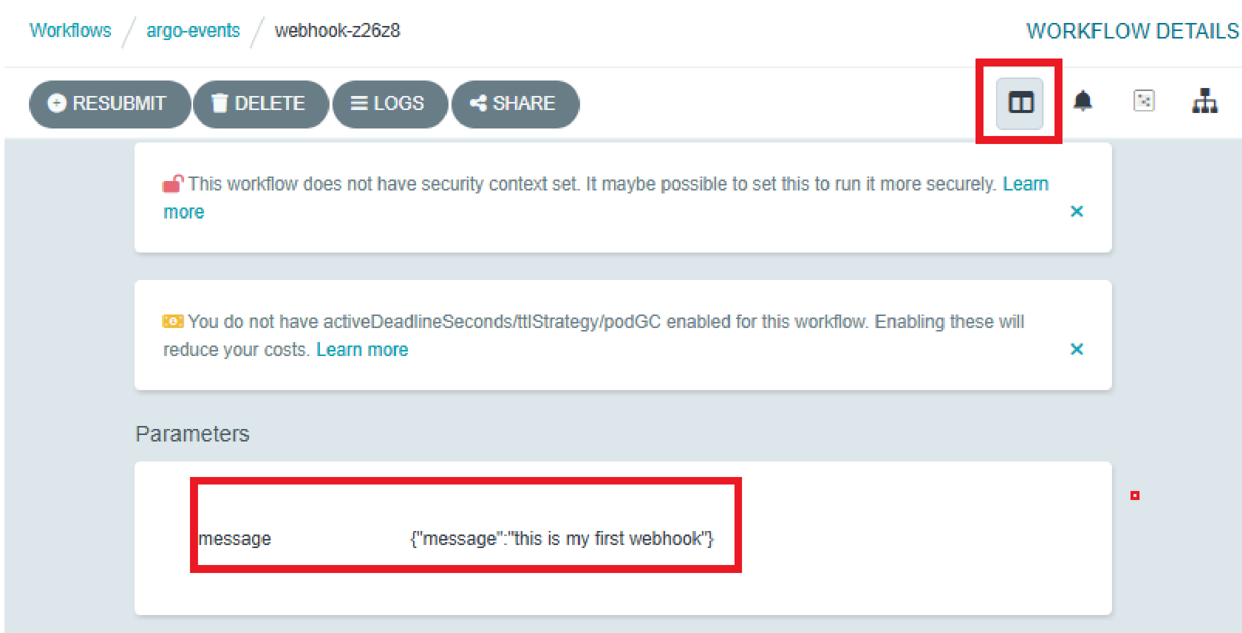
Refresh the UI of Argo Workflows. You should see a new workflow that is processing.

Refresh again after a few minutes and you should see the following screen with a completed workflow:



Workflow is shown in Argo Workflows UI

Click on the name of the workflow (e.g., **webhook-rf66b**). In the new window, select the webhook (1), select Inputs/Outputs (2) and under parameters you see the message of the curl we have sent before.



Message of curl shown in the workflow

We successfully implemented a lab setup integrating Argo Workflows and Argo Events within a Kubernetes environment, demonstrating the efficiency of event-driven automation combined with workflow management.

The key achievement was using a webhook event to initiate an Argo Workflow, showcasing the system's capability to respond to events dynamically.

In our next lab, we plan to extend this setup to connect with an external system.

Lab 6.2. Integrating Argo Events with External Systems

This lab is designed to expand your experience with Argo Events by integrating it with Apache Pulsar. After having previously triggered Argo Events workflows using a simple HTTP request, you will now use Pulsar, a distributed messaging system, as a different event source.

Objectives

- Gain practical experience in setting up and triggering workflows in Argo Events using messages from external systems.

Prerequisites

- A Kubernetes cluster is available with the components installed from Lab 6.1.
- kubectl is installed and configured to communicate with your Kubernetes cluster.
- Internet access for downloading necessary files and packages.

Use Apache Pulsar with Argo Events

1. Triggering a Workflow with Pulsar

Deploy Apache Pulsar in your cluster with the following command:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/lftraining/LFS256-code/main/argoeven
ts/pulsar.yaml
```

Check the status of the Pulsar pod and note the name. Run the command below:

```
kubectl get pods -n argo-events
```

Next, we need to port forward the Pulsar pod to enable direct communication between your local machine and the Pulsar service running in the Kubernetes cluster. This step is crucial for Argo Events to interact with Pulsar for triggering workflows. We do that with the following command:

```
kubectl -n argo-events port-forward <POD NAME OF PULSAR> 6650:6650
```

Replace **`POD NAME OF PULSAR`** with the actual name of the Pulsar pod obtained from the **`kubectl get pods`** command.

Set up the event source for Argo Events to listen to Pulsar messages. This configures Argo Events to connect and listen to Pulsar. Run the command below:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/event-sources/pulsar.yaml
```

Deploy the sensor for reacting to Pulsar events as defined by the YAML below:

```
cat <<EOF | kubectl -n argo-events apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Sensor
metadata:
  name: pulsar
spec:
  template:
    serviceAccountName: operate-workflow-sa
  dependencies:
    - name: test-dep
      eventSourceName: pulsar
      eventName: example
  triggers:
    - template:
        name: workflow-trigger
        k8s:
          operation: create
          source:
            resource:
              apiVersion: argoproj.io/v1alpha1
              kind: Workflow
              metadata:
                generateName: pulsar-wf-
              spec:
                entrypoint: cowsay
                arguments:
```

```
                parameters:
                  - name: message
                    # value will get overridden by the event payload
                    value: hello world
              templates:
                - name: cowsay
                  inputs:
                    parameters:
                      - name: message
                  container:
                    image: rancher/cowsay:latest
                    command: [cowsay]
                    args: ["{{inputs.parameters.message}}"]
        parameters:
          - src:
              dependencyName: test-dep
              dataKey: body
            dest: spec.arguments.parameters.0.value
EOF
```

This sets up the actions to be taken in response to events detected by Argo Events:

Now, everything is set up to trigger the event. To interact with the Pulsar pod, use the following commands:

```
kubectl -n argo-events get pods
```

```
kubectl -n argo-events exec -it <NAME OF PULSAR POD> -- /bin/bash
```

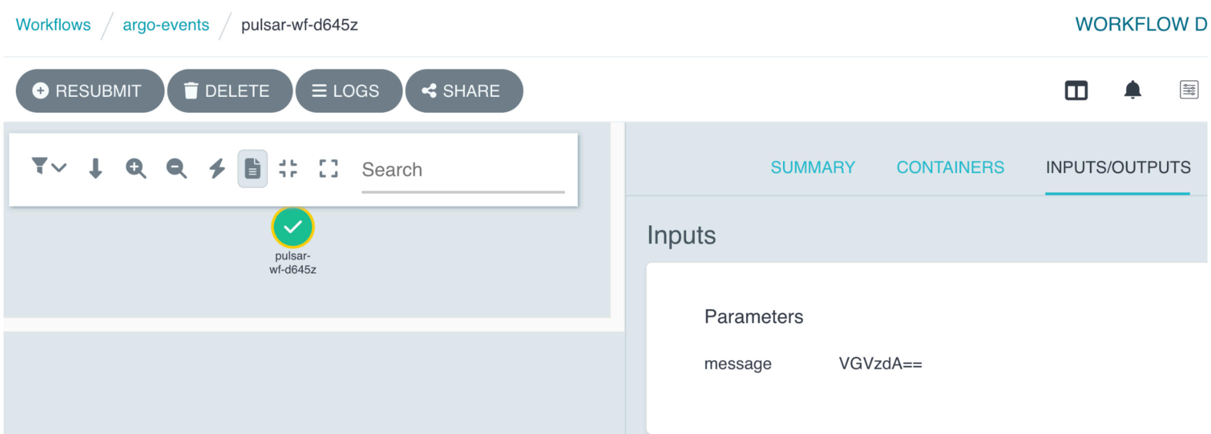
Inside the Pulsar pod, navigate to the `bin` directory and send a test message:

```
cd bin
```

```
./pulsar-client produce test --messages "Test"
```

2. Inspecting the Triggered Workflow

After sending the "Test" message via Pulsar, it triggers an Argo workflow. In the Argo UI, you can see the message in the workflow the same as in the first lab.



Message of Pulsar shown in the workflow

You can also see the output from the pod in the cluster too.

First, find the **`workflow`** pod (it should have `` `-wf` `` in the suffix. Run the following command:

```
$ kubectl get pods -n argo-events | grep pulsar
```

Output:

```
pulsar-758c47fc7-mpgd7                      1/1     Running     0
5m52s

pulsar-eventsource-ql95p-77b5bb695b-wv9q6   1/1     Running     0
4m39s

pulsar-sensor-rpqzd-df45d7dbc-drx7f         1/1     Running     0
4m17s

pulsar-wf-91xlv                             0/2     Completed   0
3m10s
```

Then check the logs with the following command:

```
$ kubectl logs -n argo-events pulsar-wf-91xlv
```

Output:

```
 __________
< VGVzdA== >
 ----------
```

```
        \   ^__^
         \  (oo)\_______
            (__)\       )\/\
                ||----w |
                ||     ||
time="2025-11-04T04:22:54.409Z" level=info msg="sub-process exited"
argo=true error="<nil>"
```

However, this message is encoded in Base64, a common practice in Apache Pulsar for efficient and reliable data serialization and transmission. To read the message correctly, you'll need to decode it from Base64. Run the command below:

```
$ echo "VGVzdA==" | base64 -d
```

Output:

```
Test
```

In this lab, we focused on the integration of Argo Events with Apache Pulsar, demonstrating how Argo Events can be configured to interact with external messaging systems. This exercise highlights Argo Events' versatility in integrating with different external systems, enhancing its capability to manage complex, event-driven architectures in cloud-native environments.